

**ΟΙΚΟΝΟΜΙΚΟ
ΠΑΝΕΠΙΣΤΗΜΙΟ
ΑΘΗΝΩΝ**



**ATHENS UNIVERSITY
OF ECONOMICS
AND BUSINESS**

**ΤΜΗΜΑ ΠΛΗΡΟΦΟΡΙΚΗΣ
ΜΕΤΑΠΤΥΧΙΑΚΟ ΔΙΠΛΩΜΑ ΕΙΔΙΚΕΥΣΗΣ (MSc)
στην ΑΝΑΠΤΥΞΗ & ΑΣΦΑΛΕΙΑ ΠΛΗΡΟΦΟΡΙΑΚΩΝ
ΣΥΣΤΗΜΑΤΩΝ**

ΔΙΠΛΩΜΑΤΙΚΗ ΕΡΓΑΣΙΑ

**Αξιολόγηση και Ανάπτυξη
Νέων Πολιτικών Αντικατάστασης Κρυφής Μνήμης:
Η Πολιτική Histogram-based LRU**

Δρίβα Ελένη

P3312106

ΑΘΗΝΑ, ΜΑΡΤΙΟΣ 2025

Περίληψη

Στα σύγχρονα υπολογιστικά συστήματα με όλο αυξανόμενες απαιτήσεις για αποδοτικότητα και ταχύτητα, η αποδοτική ανάκτηση δεδομένων αποτελεί κρίσιμο παράγοντα. Η κρυφή μνήμη, ούσα περιορισμένης χωρητικότητας, καθιστά απαραίτητη τη σωστή επιλογή και διαχείριση των δεδομένων που θα παραμείνουν σε αυτήν. Η επιλογή αυτή πραγματοποιείται μέσω πολιτικών αντικατάστασης, με τον αλγόριθμο LRU (Least Recently Used) να αποτελεί έναν από τους πιο διαδεδομένους.

Στόχος της παρούσης εργασίας είναι η ανάπτυξη μίας νέας πολιτικής αντικατάστασης κρυφής μνήμης Histogram-based LRU, με σκοπό την ορθότερη αξιολόγηση της χρησιμότητας των δεδομένων, βασισμένη στη χρονική εγγύτητα αλλά και στη συχνότητα προσπέλασης των δεδομένων, μετριάζοντας τους περιορισμούς που συνδέονται με την κλασική LRU. Αρχικά αποτελέσματα δείχνουν ότι μπορούμε να επιτύχουμε έως και **52.3%** λιγότερα cache misses, άρα και να μειώσουμε το χρόνο απόκρισης του συστήματος συγκριτικά με την LRU.

Λέξεις-Κλειδιά

κρυφή μνήμη, redis, πολιτικές αντικατάστασης, LRU, ιστόγραμμα, απόδοση κρυφής μνήμης

Abstract

In modern computing systems, where demands for efficiency and speed continue to rise, optimizing data retrieval is a critical challenge. Given the limited capacity of cache memory, it is essential to carefully select and manage the data that remain in the cache. This selection is governed by replacement policies, with the Least Recently Used (LRU) algorithm being one of the most widely adopted approaches.

This study introduces the Histogram-based LRU, an enhanced variant of the LRU algorithm designed to improve data usefulness evaluation by incorporating both temporal locality and access frequency. By addressing key limitations of the traditional LRU, our approach aims to enhance cache performance. Preliminary results indicate that the proposed method can reduce cache misses by up to **52.3%**, leading to an overall decrease in response time compared to the conventional LRU strategy.

Keywords

cache memory, redis, replacement policies, LRU, histogram, cache performance

Ευχαριστίες

Θα ήθελα να εκφράσω την ειλικρινή μου εκτίμηση στην καθηγήτριά μου Βάνα Καλογεράκη για την καθοδήγηση, τη βοήθεια και την υποστήριξή της κατά τη διάρκεια της εκπόνησης της παρούσας εργασίας. Η διδασκαλία και η στήριξή της υπήρξαν ανεκτίμητες και μου προσέφεραν πολύτιμες γνώσεις και εμπειρίες.

Ένα θερμό ευχαριστώ στην οικογένειά μου για την αμέριστη υποστήριξη, την κατανόηση και τη συνεχή βοήθεια που μου παρείχαν καθ' όλη τη διάρκεια των σπουδών μου.

Πίνακας Περιεχομένων

Περίληψη	2
Abstract	3
Ευχαριστίες	4
Λίστα Συντομογραφιών	8
Λίστα Εικόνων	9
Λίστα Πινάκων	10
Λίστα Τύπων	11
Κεφάλαιο 1: Εισαγωγή	12
1.1 Παρουσίαση Προβλήματος	12
1.2 Στόχοι της Εργασίας	13
1.3 Δομή της Εργασίας	13
Κεφάλαιο 2: Θεωρητικό Υπόβαθρο	15
2.1 Η χρησιμότητα της Μνήμης	15
2.2 Εισαγωγή στη Redis	16
2.3 Αντικατάσταση και Πολιτικές Αντικατάστασης στη Redis	17
2.3.1 Πολιτικές Αντικατάστασης	17
2.3.2 Λειτουργία Μηχανισμού Αντικατάστασης	18
2.4 Πολιτική LRU και Περιορισμοί	19
2.4.1 Approximated LRU αλγόριθμος	19
2.4.1.1 Πείραμα Εφαρμογής του Approximated LRU	19
2.4.2 Περιορισμοί της Πολιτικής LRU	20
2.5 Άλλες Έρευνες και Δημοσιεύσεις	22
2.5.1 ESCAPE: Elastic Caching for Big Data Systems	22
2.5.2 Penalty- and Locality-aware Memory Allocation in Redis Using Enhanced AET	22
2.5.3 Multi-Tenant In-memory Key-Value Cache Partitioning Using Efficient Random Sampling-Based LRU Model	23
2.5.4 MemSC: A Scan-Resistant and Compact Cache Replacement Framework for Memory-Based Key-Value Cache Systems	24
2.5.5 GD-wheel: A cost-aware replacement policy for key-value stores	24
Κεφάλαιο 3: Μεθοδολογία	25
3.1 Σχεδίαση και Υλοποίηση του Αλγορίθμου	25
3.1.1 Αρχές Λειτουργίας	25
3.1.2 Δυναμικός Πίνακας για την αποθήκευση των τελευταίων X timestamps πρόσβασης	26

3.1.3 Μετρητής για την καταγραφή της Συχνότητας Πρόσβασης σε δεδομένα	27
3.1.4 Μετρικές mean και standard deviation.....	28
3.1.5 Υπολογισμός παραθύρων ηλικίας	29
3.1.6 Υπολογισμός τελικού Score	31
3.2 Περιβάλλον και Παραμετροποίηση	33
3.2.1 Υποδομή και Περιβάλλον Εργασίας	33
3.2.2 Υλοποίηση και Επικοινωνία.....	33
3.2.3 Εργαλεία Πειραμάτων και Παρακολούθησης	34
Κεφάλαιο 4: Πειραματική Αξιολόγηση.....	35
4.1 Random Σενάριο (LRU vs HLRU)	37
4.2 Σενάριο Ευνοϊκό σε LRU με $f > 1$ (LRU vs HLRU)	39
4.3 Σενάριο LRU με $f = 1$ (LRU vs HLRU)	41
4.4 Σενάριο Hot & Cold (LRU vs HLRU).....	43
4.5 Σενάριο Σειριακής Σάρωσης (LRU vs HLRU).....	45
4.6 Σενάρια Σύγκρισης HLRU ($AT = \{1, 2, 3, 4, 5\}$).....	47
4.6.1 HLRU ($AT = \{1, 2, 3, 4, 5\}$) - Rate.....	48
4.6.2 HLRU ($AT = \{1, 2, 3, 4, 5\}$) - Used Memory vs Max Memory	49
4.6.3 HLRU ($AT = \{1, 2, 3, 4, 5\}$) - CPU Usage	50
4.6.4 HLRU ($AT = \{1, 2, 3, 4, 5\}$) - Cache Miss	51
4.6.5 HLRU ($AT = \{1, 2, 3, 4, 5\}$) - Υπολογισμοί Μνήμης.....	52
Κεφάλαιο 5: Παρουσίαση και Ανάλυση Αποτελεσμάτων.....	55
5.1 Παρατηρήσεις για όλα τα Σενάρια.....	55
5.1.1 Διαφοροποιήσεις Πειραμάτων για Ίδια Σενάρια.....	55
5.1.2 Χρήση της CPU	55
5.2 Αποτελέσματα - Random Σενάριο	56
5.3 Αποτελέσματα - Σενάριο Ευνοϊκό σε LRU με $f > 1$	56
5.4 Αποτελέσματα - Σενάριο Ευνοϊκό σε LRU με $f = 1$	57
5.5 Αποτελέσματα - Σενάριο Hot & Cold.....	57
5.6 Αποτελέσματα - Σενάριο Σειριακής Σάρωσης.....	58
5.7 Αποτελέσματα - HLRU ($AT = \{1, 2, 3, 4, 5\}$).....	59
5.7.1 Αξιολόγηση και Use Cases HLRU ($AT = \{1, 2, 3, 4, 5\}$).....	61
5.8 Συμπεράσματα	62
Κεφάλαιο 6: Περιορισμοί και Μελλοντική Έρευνα	63
6.1.1 Εκτέλεση σε Παραγωγικά Περιβάλλοντα	63
6.1.2 Δυσκολία στην Επιλογή της Βέλτιστης Τιμής AT	63

Κεφάλαιο 7: Επίλογος	64
Βιβλιογραφία	65
Appendix - Code	66

Λίστα Συντομογραφιών

AT	Access Time
HLRU	Histogram-based LRU
LRU	Least Recently Used
LFU	Least Frequently Used
TTL	Time To Live
ΒΔ	Βάση Δεδομένων

Λίστα Εικόνων

Figure 1: System Design with Cache.....	16
Figure 2: Approximated LRU Experiments.....	19
Figure 3: Ιστόγραμμα.....	31
Figure 4: Random Scenario - Rate.....	38
Figure 5: Random Scenario - Used Memory vs Max Memory	38
Figure 6: Random Scenario - CPU Usage	38
Figure 7: Random Scenario - Cache Miss Count.....	38
Figure 8: LRU-Favoured Scenario - Rate	40
Figure 9: LRU-Favoured Scenario - Used Memory vs Max Memory.....	40
Figure 10: LRU-Favoured Scenario - CPU Usage	40
Figure 11: LRU-Favoured Scenario - Cache Miss Count.....	40
Figure 12: LRU ($F = 1$) - Rate	42
Figure 13: LRU ($F = 1$) - Used Memory vs Max Memory	42
Figure 14: LRU ($F = 1$) - CPU Usage.....	42
Figure 15: LRU ($F = 1$) - Cache Miss Count.....	42
Figure 16: Hot & Cold - Rate	44
Figure 17: Hot & Cold - Used Memory vs Max Memory	44
Figure 18: Hot & Cold - CPU Usage	44
Figure 19: Hot & Cold - Cache Misses Count.....	44
Figure 20: Scan Scenario - Rate.....	46
Figure 21: Scan Scenario - Used Memory vs Max Memory	46
Figure 22: Scan Scenario - CPU Usage	46
Figure 23: Scan Scenario - Cache Miss Count	46
Figure 24: HLRU ($AT = \{1, 2, 3, 4, 5\}$) - Rate	48
Figure 25: HLRU ($AT = \{1, 2, 3, 4, 5\}$) - Used Memory vs Max Memory	49
Figure 26: HLRU ($AT = \{1, 2, 3, 4, 5\}$) - CPU Usage.....	50
Figure 27: HLRU ($AT = \{1, 2, 3, 4, 5\}$) - Cache Miss.....	51
Figure 28: HLRU - Ποσοστό Αύξησης Μνήμης για σταθερό αριθμό αντικειμένων συναρτήσσει μεγέθους.	54
Figure 29: Code - Δημιουργία HLRU variables στο redisObject (access_times and num_accesses).....	66
Figure 30: Code - Δυναμικό Configuration (history-count, histogram-window-weights)	66
Figure 31: Code – Υπολογισμός Στατιστικών Δεδομένων (Mean, Standard Deviation)	67
Figure 32: Ορισμός Δυναμικών Παραθύρων Ιστογράμματος.	67
Figure 33: Code - Υπολογισμός τελικού score.....	68

Λίστα Πινάκων

Table 1: HLRU - Μνήμη ανά Κατηγορία Αντικειμένου.	52
Table 2: Μνήμη για σταθερό αριθμό αντικειμένων ανάλογα με το μέγεθος.....	53
Table 3: HLRU - Ποσοστό Αύξησης Μνήμης συναρτήσει μεγέθους.	53
Table 4: Σενάριο Ευνοϊκό σε LRU με $f > 1$ - Ποσοστό Αύξησης Μνήμης.....	56
Table 5: Σενάριο Hot & Cold - Ποσοστό Αύξησης Cache Misses.....	57
Table 6: Σενάριο Σειριακής Σάρωσης - Ποσοστό Αύξησης Cache Misses.....	58
Table 7: HLRU ($AT = \{1, 2, 3, 4, 5\}$) - Συγκεντρωτικός Πίνακας	59
Table 8: Αξιολόγηση και Use Cases HLRU βάσει AT.....	61

Λίστα Τύπων

Formula 1: Definition of age.....	28
Formula 2: Mean Calculation	28
Formula 3: Standard Deviation Calculation	28
Formula 4: Total (Eviction) Score Calculation.....	31

Κεφάλαιο 1: Εισαγωγή

1.1 Παρουσίαση Προβλήματος

Σήμερα, η χρήση και η αποδοτική διαχείριση της κρυφής μνήμης είναι κρίσιμη για τη βελτιστοποίηση της απόδοσης σε συστήματα υψηλής διαθεσιμότητας και ταχύτητας. Στα σύγχρονα υπολογιστικά συστήματα, η κρυφή μνήμη διαδραματίζει έναν κεντρικό ρόλο, επιτρέποντας την ταχύτερη πρόσβαση σε δεδομένα που χρησιμοποιούνται συχνά και μειώνοντας το φόρτο στους κύριους αποθηκευτικούς πόρους. Ωστόσο, η κρυφή μνήμη είναι περιορισμένης χωρητικότητας και η επιλογή των δεδομένων που βρίσκονται εκεί είναι μείζονος σημασίας για την αποδοτικότητα του συστήματος. Αυτή διαχείριση της κρυφής μνήμης ρυθμίζεται από τις πολιτικές αντικατάστασης (eviction policies), οι οποίες είναι αλγόριθμοι που αποφασίζουν ποια δεδομένα είναι πιο χρήσιμα και πρέπει να διατηρηθούν σε αυτήν.

Η Redis, ως ένα από τα πιο δημοφιλή συστήματα μνήμης, αξιοποιεί διάφορες πολιτικές αντικατάστασης για να βελτιστοποιήσει την αποδοτικότητά της. Η πολιτική LRU (Least Recently Used) είναι ευρέως χρησιμοποιούμενη λόγω της απλότητας και της αποτελεσματικότητάς της σε πολλές περιπτώσεις. Η πολιτική LRU θεωρεί τα δεδομένα που δεν έχουν χρησιμοποιηθεί για το μεγαλύτερο διάστημα ως τα λιγότερο χρήσιμα, και τα απομακρύνει όταν χρειάζεται χώρος για νέα δεδομένα. Ωστόσο, παρά τα πλεονεκτήματά της, η πολιτική αυτή παρουσιάζει σημαντικά μειονεκτήματα.

Για παράδειγμα, η LRU μπορεί να αποδειχθεί λιγότερο αποτελεσματική σε σενάρια όπου πραγματοποιούνται σαρώσεις μέρους ή όλων των δεδομένων, όπου η μνήμη θα καταλήξει να περιέχει δεδομένα χαμηλής αξίας, καθώς αυτά θα έχουν εκτοπιστεί. Επιπλέον, σε περιβάλλοντα με μικτά και bursty μοτίβα πρόσβασης, η πολιτική αυτή μπορεί να μην προσφέρει την καλύτερη απόδοση.

Για να αντιμετωπιστούν αυτά τα ζητήματα, προτείνουμε μια νέα πολιτική αντικατάστασης: την histogram-based LRU. Αυτή η πολιτική αποσκοπεί στη βελτίωση της απόδοσης μέσω της δημιουργίας ενός ιστογράμματος για την παρακολούθηση της κατανομής των προσβάσεων σε διαφορετικά χρονικά διαστήματα.. Ο αλγόριθμος δεν στηρίζεται αποκλειστικά στη χρονική σειρά πρόσβασης, όπως η LRU, αλλά βασίζεται τόσο στη συχνότητα όσο και στη χρονική εγγύτητα των προσβάσεων, παραμένοντας ανεπηρέαστος σε φαινόμενα cache pollution.

1.2 Στόχοι της Εργασίας

Στόχος της παρούσης είναι η ανάπτυξη και αξιολόγηση ενός νέου αλγορίθμου αντικατάστασης κρυφής μνήμης, του histogram-based LRU στο περιβάλλον της Redis, επιδιώκοντας την αύξηση της αποδοτικότητας αξιολογώντας πιο σωστά τη χρησιμότητα των δεδομένων.

Θα παρουσιαστεί αρχικά η σημασία της μνήμης, καθώς και η πιο δημοφιλής υλοποίησή της, το σύστημα Redis, ως προς τον τρόπο λειτουργίας του. Θα μελετήσουμε τον τρόπο λειτουργίας της πολιτικής LRU μαζί με τα πλεονεκτήματά της, αλλά και τους περιορισμούς της.

Θα συγκρίνουμε τους δύο αλγορίθμους σε διαφορετικά σενάρια φόρτου και μοτίβου ανάκτησης δεδομένων, αξιολογώντας τα αποτελέσματα αυτών βάσει δεικτών όπως ο χρόνος απόκρισης του συστήματος. Θα παραμετροποιήσουμε την υλοποίησή μας για να την αξιολογήσουμε με τις διάφορες τιμές της.

Τέλος, βάσει των αποτελεσμάτων των συγκρίσεων θα εξάγουμε αποτελέσματα και συμπεράσματα, από τα οποία θα προκύψουν και προτάσεις για βελτίωση αλλά και πεδία μελλοντικής έρευνας.

1.3 Δομή της Εργασίας

Η δομή της παρούσης εργασίας είναι οργανωμένη ως εξής:

Στο **Κεφάλαιο 2**, θα εξετάσουμε τη χρησιμότητα της κρυφής μνήμης και την εφαρμογή της στη Redis. Στην αρχή, θα αναλύσουμε την αναγκαιότητα της κρυφής μνήμης και τα οφέλη που προσφέρει. Στη συνέχεια, θα παρουσιαστεί η Redis, περιγράφοντας τα βασικά χαρακτηριστικά της και τον τρόπο λειτουργίας της ως σύστημα κρυφής μνήμης. Θα ακολουθήσει μια ανασκόπηση των πολιτικών αντικατάστασης που διαθέτει η Redis, εστιάζοντας ειδικά στη πολιτική LRU με τις παραλλαγές της. Τέλος, θα αναδείξουμε τα τρωτά της σημεία και τους περιορισμούς της.

Στο **Κεφάλαιο 3**, θα παρουσιάσουμε τη μεθοδολογία, δηλαδή την ανάπτυξη και υλοποίηση της νέας πολιτικής αντικατάστασης histogram-based LRU. Θα περιγράψουμε τη λογική του αλγορίθμου, τη λειτουργία του ιστογράμματος, τη διαδικασία ενσωμάτωσης του νέου αλγορίθμου στη Redis, και τις στρατηγικές βελτιστοποίησης που εφαρμόστηκαν.

Στο **Κεφάλαιο 4**, θα επικεντρωθούμε στη σύγκριση και αξιολόγηση του histogram-based LRU σε σχέση με την παραδοσιακή πολιτική LRU. Θα παρουσιαστούν τα πειράματα που διεξήχθησαν μαζί με τις παραμέτρους τους, καθώς και συμπεράσματα που προκύπτουν από την αξιολόγησή τους. Η αξιολόγηση θα περιλαμβάνει συγκρίσεις μεταξύ των δύο πολιτικών με βάση δείκτες όπως η αποδοτικότητα και η ακρίβεια αντικατάστασης. Θα παρουσιαστούν, επίσης, πειράματα σύγκρισης της HLRU με παραμετροποιήσεις δυναμικών παραμέτρων. Θα γίνουν, τέλος, αναλυτικοί υπολογισμοί για την επιπλέον μνήμη που χρειάζεται η πολιτική HLRU.

Στο **Κεφάλαιο 5**, θα παρουσιαστούν τα συμπεράσματα της έρευνας. Αρχικά, θα εξαχθούν πορίσματα που αφορούν όλα τα πειράματα, και στη συνέχεια πορίσματα που αφορούν μεμονωμένα κάθε σενάριο. Θα γίνει αξιολόγηση της συνολικής αποδοτικότητας της HLRU και θα φτιαχτεί ένας οδηγός χρήσης συναρτήσει των διαφορετικών παραμέτρων της.

Στο **Κεφάλαιο 6**, θα αναγνωριστούν οι περιορισμοί της έρευνάς μας, αλλά και προτάσεις για το μέλλον.

Στο **Κεφάλαιο 7**, θα συνοψίσουμε τα συμπεράσματα της έρευνάς μας.

Κεφάλαιο 2: Θεωρητικό Υπόβαθρο

2.1 Η χρησιμότητα της Μνήμης

Στη σύγχρονη ανάπτυξη λογισμικού, η απόδοση ενός συστήματος παίζει καθοριστικό ρόλο στην εμπειρία του χρήστη και στη συνολική αποδοτικότητα των εφαρμογών. Η κρυφή μνήμη (cache) είναι μια βασική τεχνική που χρησιμοποιείται για τη βελτίωση της απόδοσης με την προσωρινή αποθήκευση δεδομένων που προσπελούνται συχνά, σε μνήμη υψηλής ταχύτητας, όπως η RAM. Επιπλέον, για να ανταποκρίνονται στις απαιτήσεις ενός αυξανόμενου αριθμού χρηστών ή όγκου δεδομένων, τα συστήματα πρέπει να είναι και κλιμακώσιμα (scalable), δηλαδή να μπορούν να επεκτείνονται ομαλά χωρίς να μειώνεται η απόδοσή τους.

Η χρήση κρυφής μνήμης συμβάλλει καθοριστικά σε αυτό, καθώς επιτρέπει την ταχύτερη εξυπηρέτηση αιτημάτων, μειώνοντας ταυτόχρονα τον φόρτο στους κεντρικούς πόρους του συστήματος.

Η βασική ιδέα πίσω από την κρυφή μνήμη είναι η αποθήκευση δεδομένων που είναι πιθανό να χρησιμοποιηθούν ξανά στο άμεσο μέλλον σε μια ταχύτερη, αλλά περιορισμένης χωρητικότητας δομή.

Έτσι, όταν τα δεδομένα αυτά ζητηθούν, μπορούν να ανακτηθούν πολύ γρηγορότερα από ό,τι αν έπρεπε να ανακτηθούν από την κύρια αποθήκευση, όπως ένας σκληρός δίσκος ή μια βάση δεδομένων.

Αυτό έχει ως αποτέλεσμα τη μείωση του χρόνου απόκρισης των εφαρμογών και του φόρτου στην κύρια βάση δεδομένων, επιτρέποντας την εξυπηρέτηση των αιτημάτων με μεγαλύτερη ταχύτητα.

Δημοφιλή συστήματα caching αποτελούν η Redis, το Memcached, η Ehcache, και το Hazelcast.

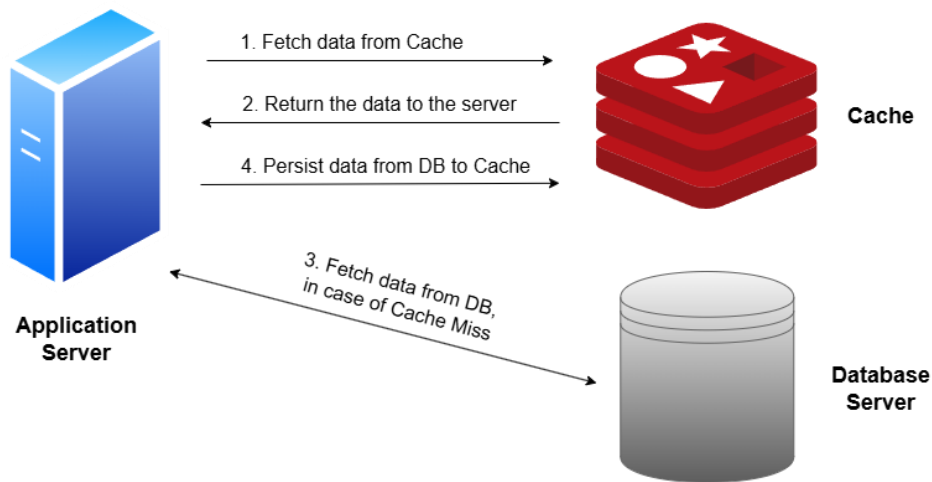


Figure 1: System Design with Cache

2.2 Εισαγωγή στη Redis

Η **Redis (Remote Dictionary Server)** είναι μια ανοιχτού κώδικα NoSQL βάση δεδομένων βασισμένη σε μια δομή μορφής key-value. Μπορεί να χρησιμοποιηθεί σαν κρυφή μνήμη, βάση δεδομένων, message broker και μηχανή streaming υποστηρίζοντας πολλαπλές δομές δεδομένων όπως strings, hashes, lists, sorted sets, bitmaps, hyperlogs, geospatial indexes και streams.

Η Redis χρησιμοποιείται ευρέως ως σύστημα κρυφής μνήμης (caching) για να μειώνει το φόρτο στις βάσεις δεδομένων και να βελτιώνει την απόδοση των εφαρμογών. Λόγω της υψηλής απόδοσής της χρησιμοποιείται σε real time εφαρμογές μεγάλου φόρτου με απαιτητικές ανάγκες απόκρισης, όπως το gaming, τα financial services και το IoT.

Η επίτευξη υψηλής απόδοσης με χαμηλή καθυστέρηση (latency) και υψηλό ρυθμό διακίνησης δεδομένων (throughput) καταδεικνύουν τη σημασία της Redis στην σύγχρονη ανάπτυξη λογισμικού, όπου η ταχύτητα πρόσβασης στα δεδομένα αποτελεί καθοριστικό παράγοντα για την επιτυχία των εφαρμογών.

Θεωρείται ότι είναι η πιο δημοφιλής NoSQL βάση δεδομένων και μία από τις πιο γνωστές βάσεις δεδομένων συνολικά. Στους χρήστες της Redis συμπεριλαμβάνονται οι εταιρείες Twitter, Airbnb, Amazon, OpenAI και Tinder.

2.3 Αντικατάσταση και Πολιτικές Αντικατάστασης στη Redis

2.3.1 Πολιτικές Αντικατάστασης

Σε συστήματα κρυφής μνήμης όπως η Redis, η μνήμη είναι ένας πεπερασμένος πόρος. Όταν η μνήμη φτάσει το 100% του ορίου μνήμης (*maxmemory*) της, η Redis αποφασίζει ποια δεδομένα θα απομακρυνθούν από τη μνήμη προκειμένου να δημιουργήσει χώρο για νέα δεδομένα. Αυτή η διαδικασία είναι γνωστή ως αντικατάσταση (*eviction*), τα κριτήρια και εν τέλει η επιλογή των δεδομένων που θα αφαιρεθούν από τη μνήμη καθορίζονται από τις πολιτικές αντικατάστασης (*eviction policies*).

Η Redis διαθέτει διάφορες πολιτικές αντικατάστασης προσφέροντας τη δυνατότητα στον χρήστη να επιλέξει την πιο ωφέλιμη πολιτική που θα χρησιμοποιήσει στην εφαρμογή του:

- **noeviction:** τα νέα δεδομένα δεν αποθηκεύονται στην περίπτωση που το όριο μνήμης έχει ξεπερασθεί. Προεπιλεγμένη τιμή.
- **allkeys-lru:** πολιτική βασιζόμενη στον LRU αλγόριθμο, όπου τα δεδομένα που δεν έχουν χρησιμοποιηθεί για το μεγαλύτερο διάστημα θεωρούνται λιγότερο σημαντικά και απομακρύνονται πρώτα. Εφαρμόζεται σε όλα τα δεδομένα που βρίσκονται στη μνήμη.
- **allkeys-lfu:** Πολιτική βασιζόμενη στον αλγόριθμο LFU, όπου τα δεδομένα που έχουν ανακτηθεί λιγότερες φορές θεωρούνται λιγότερο σημαντικά και απομακρύνονται πρώτα. Εφαρμόζεται σε όλα τα δεδομένα που βρίσκονται στη μνήμη.
- **allkeys-random:** Απομακρύνονται τα δεδομένα με τυχαίο τρόπο - χωρίς κριτήριο
- **volatile-lru:** Λειτουργεί όπως η *allkeys-lru*, αλλά εφαρμόζεται μόνο σε κλειδιά που έχουν χρόνο ζωής (TTL)
- **volatile-lfu:** Λειτουργεί όπως η *allkeys-lfu*, αλλά εφαρμόζεται μόνο σε κλειδιά που έχουν χρόνο ζωής (TTL)
- **volatile-random:** Λειτουργεί όπως η *allkeys-random*, αλλά εφαρμόζεται μόνο σε κλειδιά που έχουν χρόνο ζωής (TTL)
- **volatile-ttl:** εφαρμόζεται μόνο σε κλειδιά που έχουν χρόνο ζωής και μάλιστα το λιγότερο (TTL)

Επί της ορθής επιλογής έκαστης πολιτικής, σαν γενικό κανόνα ακολουθούμε:

- **allkeys-lru:** χρησιμοποιείται όταν τα αιτήματα για την ανάκτηση των δεδομένων ακολουθούν κατανομή νόμου δύναμης (*power-law distribution*), δηλαδή ένα μέρος των

δεδομένων ζητείται πολύ πιο συχνά από το υπόλοιπο. Αποτελεί μια καλή επιλογή στην περίπτωση που κάποιος δεν είναι σίγουρος.

- **alkeys-random:** χρησιμοποιείται στην περίπτωση κυκλικής ζήτησης των κλειδιών, όπου όλα τα κλειδιά ζητούνται συνεχώς, ή όταν η κατανομή των αιτημάτων κατανέμεται ομοιόμορφα (uniform distribution).
- **volatile-ttl:** χρησιμοποιείται αν θέλουμε να δώσουμε εμείς μια βαρύτητα στη Redis σχετικά με το ποιος είναι ένας καλός υποψήφιος για eviction, χρησιμοποιώντας το πεδίο TTL με διαφορετικές τιμές κατά τη δημιουργία των δεδομένων στην cache.
- **volatile-lru & volatile-random:** είναι κυρίως χρήσιμες όταν έχουμε ένα Redis instance και θέλουμε να το χρησιμοποιούμε και για caching και για μία λίστα από κλειδιά που θέλουμε πάντα να τα βρίσκουμε στην cache.

2.3.2 Λειτουργία Μηχανισμού Αντικατάστασης

Ο τρόπος που λειτουργεί ο μηχανισμός της αντικατάστασης είναι:

- Όταν ένας client εκτελεί μια εντολή τύπου PUT, προκαλείται η προσθήκη νέων δεδομένων στη μνήμη cache.
- Η Redis παρακολουθεί συνεχώς τη χρήση της μνήμης και, όταν το ποσοστό χρήσης υπερβεί το όριο του *maxmemory*, ενεργοποιεί τη διαδικασία διαγραφής (eviction) των κλειδιών, ακολουθώντας την επιλεγμένη πολιτική αντικατάστασης.
- Η διαδικασία επαναλαμβάνεται με κάθε περαιτέρω προσθήκες.
- Με αυτόν τον τρόπο, η μνήμη cache συνεχώς ξεπερνά το όριο του *maxmemory*, το οποίο μετά τη διαγραφή των δεδομένων επανέρχεται κάτω από το καθορισμένο όριο.

2.4 Πολιτική LRU και Περιορισμοί

2.4.1 Approximated LRU αλγόριθμος

Η Redis δεν υλοποιεί ακριβώς τον αλγόριθμο LRU, καθώς δεν είναι σε θέση να επιλέξει με ακρίβεια τον καλύτερο υποψήφιο για αντικατάσταση, δηλαδή το κλειδί που έχει χρησιμοποιηθεί πιο παλιά. Αντ' αυτού, ακολουθεί μια προσεγγιστική παραλλαγή της LRU, δειγματοληπτώντας ένα μικρό σύνολο κλειδιών και αντικαθιστώντας το κλειδί που είναι το πιο "παλιό" (σύμφωνα με τον χρόνο αναζήτησης) από τα κλειδιά που ελέγχθηκαν. Η ακρίβεια του αλγορίθμου μπορεί να παραμετροποιηθεί, επιλέγοντας τον αριθμό των δειγμάτων που θα εξεταστούν σε κάθε προσπάθεια διαγραφής μέσω της παραμέτρου *maxmemory-samples*.

Ο λόγος για τον οποίο η Redis δεν υιοθετεί τον πλήρη LRU αλγόριθμο σχετίζεται με το υψηλό κόστος σε μνήμη και υπολογιστική ισχύ (CPU), που συνεπάγεται η συνεχής παρακολούθηση και διαχείριση των χρόνων πρόσβασης όλων των κλειδιών.

2.4.1.1 Πείραμα Εφαρμογής του Approximated LRU

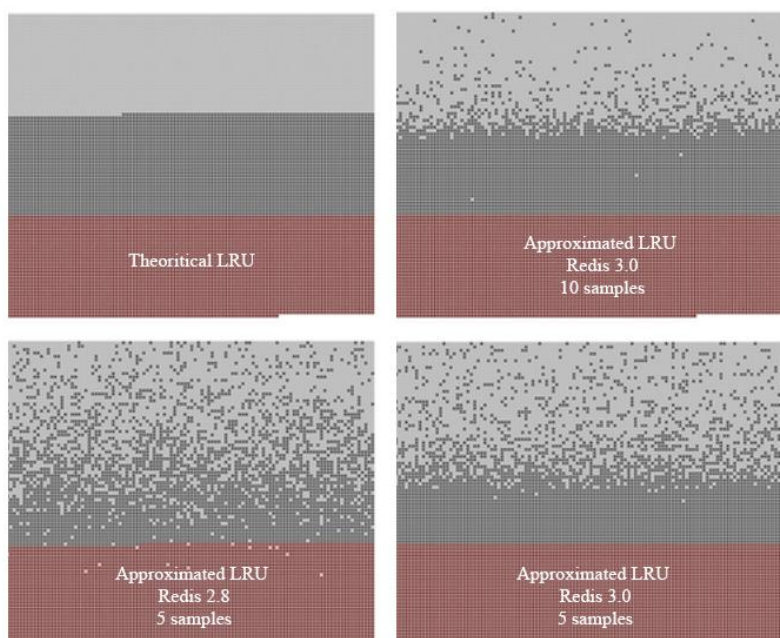


Figure 2: Approximated LRU Experiments

Από την Redis 3.0 και μετά, ο αλγόριθμος της Redis δειγματοληπτεί από ένα καλό σύνολο δεδομένων τους υποψηφίους προς απομάκρυνση. Αυτό βελτιώνει την απόδοση του αλγορίθμου, κάνοντας τον να είναι φέρνει πολύ κοντινά αποτελέσματα με την κανονική LRU.

Στο πείραμα αυτό, η Redis γεμίζει τη μνήμη cache με έναν δεδομένο αριθμό κλειδιών. Στη συνέχεια, όλα τα κλειδιά αναζητούνται από το πρώτο μέχρι το τελευταίο. Σύμφωνα με τον αλγόριθμο LRU, τα πρώτα κλειδιά είναι οι καλύτεροι υποψήφιοι για αντικατάσταση, καθώς έχουν χρησιμοποιηθεί πιο παλιά. Στη συνέχεια, προστέθηκαν 50% περισσότερα κλειδιά, προκειμένου να προκαλέσουν την αντικατάσταση των μισών από τα παλαιότερα κλειδιά.

Στο διάγραμμα, παρατηρούμε τρεις διαφορετικές ζώνες:

- Η ανοιχτόχρωμη γκρι ζώνη αντιπροσωπεύει τα κλειδιά που διαγράφηκαν.
- Η σκουρόχρωμη γκρι ζώνη περιλαμβάνει τα κλειδιά που δεν διαγράφηκαν.
- Η κόκκινη ζώνη δείχνει τα κλειδιά που προστέθηκαν.

Στη θεωρητική υλοποίηση του αλγορίθμου LRU, αναμένονται να διαγραφούν τα πρώτα μισά κλειδιά. Στην approximated-LRU της Redis, τα πιο παλιά κλειδιά πιθανότατα θα διαγραφούν, αλλά με μικρότερη ακρίβεια λόγω πιθανής μη επιλογής τους κατά την δειγματοληψία.

Όπως φαίνεται στο διάγραμμα, η Redis 3.0 με δείγμα 5 κλειδιών αποδεικνύεται πιο αποδοτική από την Redis 2.8. Βάσει των πειραμάτων της Redis, σε περιβάλλοντα όπου η πρόσβαση στα δεδομένα ακολουθεί power law κατανομή, η διαφορά μεταξύ της πραγματικής υλοποίησης LRU και της προσέγγισης της Redis είναι ελάχιστη ή και ανύπαρκτη.

2.4.2 Περιορισμοί της Πολιτικής LRU

Η LRU είναι μία από τις πιο διαδεδομένες πολιτικές αντικατάστασης λόγω της ικανότητάς της να προσαρμόζεται σε ποικίλες περιπτώσεις χρήσης, προσφέροντας μια καλή ισορροπία μεταξύ απόδοσης και αποδοτικότητας. Χρησιμοποιείται κυρίως όταν τα αιτήματα για την ανάκτηση των δεδομένων ακολουθούν κατανομή νόμου δύναμης (power-law distribution), δηλαδή όταν ένα μέρος των δεδομένων ζητείται συχνότερα από το υπόλοιπο. Αποτελεί, επίσης, μια καλή επιλογή στην περίπτωση που δεν υπάρχει σαφής προτίμηση περί επιλογής της αποδοτικότερης για την εφαρμογή πολιτική αντικατάστασης.

Ωστόσο, όπως όλες οι πολιτικές αντικατάστασης, η LRU έχει τα πλεονεκτήματα και μειονεκτήματά της. Παρά την καλή της απόδοση σε πολλές περιπτώσεις, μπορεί να αντιμετωπίσει προβλήματα όταν τα δεδομένα ακολουθούν ορισμένα μοτίβα πρόσβασης.

2.4.2.1 Περιοδικά Πρότυπα Πρόσβασης (Burst Access Patterns)

Σε περιπτώσεις που τα δεδομένα χρησιμοποιούνται περιοδικά και ενδεχομένως σε μικρές χρονικές περιόδους, η LRU μπορεί να αποτύχει να προβλέψει τις μελλοντικές ανάγκες για δεδομένα που ενδέχεται να χρειαστούν εκ νέου σύντομα. Η LRU βασίζεται αποκλειστικά στη χρονική εγγύτητα της χρήσης των δεδομένων και όχι στη συχνότητα της πρόσβασης, γεγονός που περιορίζει την αποδοτικότητά της σε τέτοιες περιπτώσεις.

2.4.2.2 Μικτά Πρότυπα Πρόσβασης (Mixed Access Patterns)

Η LRU αντιμετωπίζει δυσκολίες στη διαχείριση δεδομένων που έχουν διαφορετική συχνότητα πρόσβασης. Αυτή η πολιτική αντικατάστασης αποτυγχάνει σε περιβάλλοντα όπου κάποια δεδομένα χρησιμοποιούνται συχνά ενώ άλλα πολύ σπάνια, επειδή δεν λαμβάνει υπ' όψιν τη συχνότητα πρόσβασης αλλά μόνο τη χρονική εγγύτητα. Ως αποτέλεσμα, σημαντικά δεδομένα που δεν έχουν χρησιμοποιηθεί πρόσφατα ενδέχεται να απομακρυνθούν από τη μνήμη, χωρίς να λαμβάνεται υπ' όψιν η συνολική τους σημασία για την εφαρμογή.

2.4.2.3 Πρότυπα με Σειριακές και Πλήρεις Σαρώσεις της ΒΔ (Sequential Access Pattern)

Παρόμοια με τα μικτά πρότυπα πρόσβασης, η LRU αντιμετωπίζει προβλήματα και με τα σειριακά πρότυπα πρόσβασης λόγω της αδυναμίας της να λάβει υπ' όψιν τη συχνότητα των δεδομένων. Σε αυτές τις περιπτώσεις, κάθε νέα εισαγωγή δεδομένων στη μνήμη εκτοπίζει τα προηγούμενα, ακόμα και αν αυτά χρησιμοποιούνται συχνά ή είναι απαραίτητα για επόμενες αναζητήσεις. Η συνεχής ανανέωση της κρυφής μνήμης με δεδομένα σπανίως χρησιμοποιούμενα, οδηγεί στην ανάκτηση των δεδομένων από τη βάση δεδομένων και, κατ' επέκτασιν, σε μείωση της απόδοσης.

Η προαναφερθείσα συμπεριφορά μπορεί να συνδεθεί και με το φαινόμενο της μόλυνσης της προσωρινής μνήμης (cache pollution), φαινόμενο προερχόμενο από την αρχιτεκτονική υπολογιστών.

2.5 Άλλες Έρευνες και Δημοσιεύσεις

2.5.1 ESCAPe: Elastic Caching for Big Data Systems

Δημιουργοί: Thanasis Priovolos, Stathis Maroulis, Vana Kalogeraki (Οκτώβριος, 2019)

Η εν λόγω δημοσίευση, παρουσιάζεται το ESCAPe, ένα ελαστικό σύστημα αποθήκευσης κλειδιού-τιμής στη μνήμη, σχεδιασμένο για υψηλή απόδοση και χαμηλή καθυστέρηση. Προσφέρει έναν μηχανισμό δυναμικής προσαρμογής, προσθέτοντας ή αφαιρώντας κόμβους ώστε να ανταποκρίνεται στις μεταβαλλόμενες απαιτήσεις των εφαρμογών. Παράλληλα, ενσωματώνει μια στρατηγική κατανομής κλειδιών σε όλους τους κόμβους που μειώνει το λειτουργικό κόστος.

Το ESCAPe διαθέτει έναν ειδικό αλγόριθμο πολιτικής αντικατάστασης αντικειμένων από τη μνήμη cache, βασισμένο σε έναν σταθμισμένο συντελεστή που λαμβάνει υπ'όψιν τη συχνότητα και το μέγεθος των αντικειμένων. Ο ESCAPe Agent παρακολουθεί τη χρήση του χώρου στον κάθε κόμβο, ώστε να διατηρούνται στη μνήμη τα πιο χρήσιμα αντικείμενα, βελτιώνοντας την αναλογία επιτυχημένων προσβάσεων (hit ratio) και μειώνοντας τον χρόνο απόκρισης της εκάστοτε εφαρμογής.

Σχετικές Δημοσιεύσεις:

- *A Framework for Managing an Elastic Redis Cache*

2.5.2 Penalty- and Locality-aware Memory Allocation in Redis Using Enhanced AET

Δημιουργοί: Cheng Pan, Yingwei Luo, Xiaolin Wang, Zhenlin Wang (Ιούνιος, 2021)

Η εν λόγω δημοσίευση προτείνει μια βελτιωμένη μέθοδο διαχείρισης της μνήμης στις συστήματα προσωρινής αποθήκευσης (caching) όπως η Redis. Εστιάζει στο πώς μπορούν να συνδυαστούν η εντοπιότητα δεδομένων (locality) και η ποινή απώλειας (miss penalty) κατά την κατανομή και αντικατάσταση των δεδομένων στη μνήμη cache. Αντί να χρησιμοποιούνται παραδοσιακοί αλγόριθμοι όπως το LRU, που αγνοούν το κόστος μιας απώλειας από την cache, η pRedis προτείνει μια πιο ποσοτική προσέγγιση που βελτιστοποιεί τη χρήση μνήμης και μειώνει την καθυστέρηση (latency).

Επιπλέον, εισάγει ένα αυτόματο μηχανισμό απόθεσης και φόρτωσης δεδομένων (dump/load) για την προσαρμογή σε ημερήσιες διακυμάνσεις στη χρήση (π.χ. ώρες αιχμής και χαμηλής χρήσης). Πειράματα δείχνουν ότι η pRedis μειώνει σημαντικά τη μέση και τη μέγιστη

καθυστέρηση, υπερβαίνοντας σύγχρονες μεθόδους, όπως το Hyperbolic Caching, με μείωση καθυστερήσεων κατά 14-52% σε πραγματικούς και συνθετικούς φόρτους δεδομένων, ενώ διατηρεί χαμηλό κόστος και προβλεψιμότητα απόδοσης. Με δυναμική εναλλαγή μεταξύ της pRedis και της HC πολιτικής, ο μέσος όρος latency μειώθηκε κατά 1.1% με 45%.

Σχετικές Προηγούμενες Δημοσιεύσεις:

- *PACE: Penalty aware cache modeling with enhanced AET (Οκτώβριος, 2018)*
- *pRedis: Penalty and locality aware memory allocation in Redis (Νοέμβριος, 2019)*

2.5.3 Multi-Tenant In-memory Key-Value Cache Partitioning Using Efficient Random Sampling-Based LRU Model

Δημιουργοί: Yuchen Wang, Junyao Yang, Zhenlin Wang (Δεκέμβριος, 2023)

Το άρθρο εστιάζει στην πολιτική αντικατάστασης LRU και αναλύει γιατί η υλοποίησή της μπορεί να είναι ακριβή, καθώς απαιτεί σύνθετες δομές δεδομένων και κλειδιά για την ενημέρωση της προτεραιότητας των αντικειμένων.

Ιδιαίτερη έμφαση δίνεται στη Redis, χρησιμοποιείται μια προσεγγιστική εκδοχή της LRU, δηλαδή η approximated LRU, που επιλέγει τυχαία αντικείμενα από ένα μικρό δείγμα για να τα αντικαταστήσει, μειώνοντας το κόστος της πολιτικής. Ωστόσο, αυτή η προσέγγιση δημιουργεί μια σημαντική διαφορά στην αναλογία σφαλμάτων (miss ratio), η οποία εξαρτάται από το μέγεθος του δείγματος. Για να το διορθώσει αυτό, το άρθρο προτείνει τον αλγόριθμο KRR που μοντελοποιεί με ακρίβεια την τυχαία δειγματοληψία της LRU και προτείνει τη kRedis, ένα σύστημα δυναμικής κατανομής μνήμης, το οποίο βελτιώνει τη χρήση τοπικότητας και καθυστέρησης με δυναμική κατανομή μνήμης.

Οι αξιολογήσεις δείχνουν ότι η kRedis επιτυγχάνει έως και 50,2% μείωση στην καθυστέρηση πρόσβασης και έως και 262,8% βελτίωση στην απόδοση σε σχέση με τη Redis, ενώ συγκριτικά με τη pRedis, προσφέρει βελτιώσεις της τάξης του 24,8% στην καθυστέρηση και 61,8% στην απόδοση.

Σχετικές Προηγούμενες Δημοσιεύσεις:

- *Dynamically Configuring LRU Replacement Policy in Redis (Μάρτιος, 2021)*
- *Efficient Modeling of Random Sampling-Based LRU Cache (Οκτώβριος, 2021)*

2.5.4 MemSC: A Scan-Resistant and Compact Cache Replacement Framework for Memory-Based Key-Value Cache Systems

Δημιουργοί: Mei Li, Hong-Jun Zhang, Yan-Jun Wu, Chen Zhao (Ιανουάριος, 2017)

Αυτό το άρθρο εισάγει το MemSC, μία νέα πολιτική αντικατάστασης της μνήμης για το Memcached, η οποία είναι σχεδιασμένη να είναι αποτελεσματική σε μοτίβα σάρωσης δεδομένων (scan) όπου η παραδοσιακή πολιτική LRU υστερεί. Η MemSC χρησιμοποιεί σημαία αναφοράς πολλαπλής κλίμακας για κάθε αντικείμενο, η οποία απαιτεί μόλις λίγα bits (δύο είναι αρκετά για γενική χρήση) και υποστηρίζει πολιτικές αντικατάστασης που αντέχουν σε σάρωση. Αυτό το χαρακτηριστικό βελτιώνει την απόδοση σε περιβάλλοντα που περιλαμβάνουν μικρά δεδομένα και συχνές αιτήσεις σάρωσης. Σε πειραματικές δοκιμές, επιτυγχάνεται μείωση του cache miss ratio και της χρήσης μνήμης μέχρι και 23% και 14% αντίστοιχα, σε σχέση με την βελτιστοποιημένη πολιτική LRU του Memcached.

2.5.5 GD-wheel: A cost-aware replacement policy for key-value stores

Δημιουργοί: Conglong Li, Alan L. Cox (Απρίλιος, 2015)

Αυτό το άρθρο εισάγει τη νέα πολιτική αντικατάστασης GD-Wheel, η οποία προσαρμόζεται στις μεταβλητές δαπάνες επανυπολογισμού των δεδομένων σε μνήμη, όπως συμβαίνει σε εργαλεία όπως το Memcached και η Redis. Αντί να αγνοεί τη διαφορά κόστους μεταξύ των αποτελεσμάτων που αποθηκεύονται στη μνήμη (όπως συμβαίνει με τις παραδοσιακές πολιτικές όπως η LRU), η GD-Wheel συνδυάζει την πρόσφατη χρήση με το κόστος επανυπολογισμού για να επιλέξει ποια δεδομένα να αποθηκευτούν ή να διαγραφούν, επιτυγχάνοντας έτσι βελτιώσεις στην απόδοση. Η μελέτη δείχνει ότι η εφαρμογή της GD-Wheel μειώνει το συνολικό κόστος επανυπολογισμού κατά 90% και βελτιώνει το μέσο χρόνο απόκρισης και την καθυστέρηση του 99th percentile κατά 56% και 85% αντίστοιχα, σε σύγκριση με την παραδοσιακή LRU.

Κεφάλαιο 3: Μεθοδολογία

Κατόπιν μελέτης και κατανόησης του τρόπου λειτουργίας της LRU, θα αναπτύξουμε τη δική μας πολιτική αντικατάστασης προκειμένου να αντιμετωπίσουμε ορισμένες από τις κύριες αδυναμίες της πολιτικής LRU.

3.1 Σχεδίαση και Υλοποίηση του Αλγορίθμου

3.1.1 Αρχές Λειτουργίας

Ο histogram-based LRU αποσκοπεί στη βελτίωση της παραδοσιακής πολιτικής LRU παρακολουθώντας τη συχνότητα και τον χρόνο πρόσβασης των δεδομένων.

Ως εκ τούτου, ο αλγόριθμος εισάγει και χρησιμοποιεί δύο βασικές δομές για κάθε δεδομένο:

- έναν *μετρητή συχνότητας* που καταγράφει πόσες φορές έχει ζητηθεί το δεδομένο
- έναν *δυναμικό πίνακα* για την αποθήκευση των τελευταίων X *timestamps* πρόσβασης του δεδομένου

Σε κάθε διαδικασία αντικατάστασης υπολογίζονται κάποια στατιστικά δεδομένα για το σύνολο των δειγματοληφθέντων δεδομένων (*maxmemory-samples*):

- Υπολογίζονται οι μετρικές *mean* και *standard deviation* της ηλικίας (*age*)
- Υπολογίζεται η μικρότερη (*min*) και μεγαλύτερη (*max*) τιμή της ηλικίας (*age*)
- Υπολογίζονται *παράθυρα ηλικίας*, βάσει των παραπάνω

Για το κάθε δειγματοληφθέν αντικείμενο, υπολογίζεται ένα τελικό *score*, βάσει μιας απλής *weighted* φόρμουλας που λαμβάνει υπ' όψιν τη συχνότητα των *access times* ανά έκαστο παράθυρο. Έτσι, όταν ολοκληρωθεί η διαδικασία για όλα τα δεδομένα, το δεδομένο που είναι στην πρώτη θέση του πίνακα, απομακρύνεται ως το λιγότερο χρήσιμο.

3.1.2 Δυναμικός Πίνακας για την αποθήκευση των τελευταίων X timestamps πρόσβασης

Ο **δυναμικός πίνακας** για την αποθήκευση των τελευταίων X timestamps πρόσβασης (AT) είναι μια κεντρική συνιστώσα του αλγορίθμου Histogram-based LRU. Σκοπός του είναι να καταγράφει τις X πιο πρόσφατες χρονικές στιγμές που ένα αντικείμενο ζητήθηκε στη μνήμη. Καθώς τα δεδομένα προσπελαύνονται, τα timestamps εισάγονται στον πίνακα. Όταν ο πίνακας γεμίσει, οι παλαιότερες χρονικές στιγμές διαγράφονται μέσω κυκλικής μετατόπισης, διατηρώντας πάντα τα πιο πρόσφατα δεδομένα. Αυτή η διαδικασία επιτρέπει μια πιο ακριβή εκτίμηση της τρέχουσας χρησιμότητας των δεδομένων, βοηθώντας στη λήψη αποφάσεων για την αντικατάσταση στη μνήμη.

Το μέγεθος του εν λόγω πίνακα, και κατ'επέκτασιν του αριθμού των χρονικών στιγμών που αποθηκεύουμε, είναι *παραμετροποιήσιμο*. Για να καθοριστεί το κατάλληλο μέγεθος, πρέπει να ληφθούν υπ'όψιν διάφοροι παράγοντες, όπως το μοτίβο πρόσβασης στα δεδομένα και το σχετικό trade-off μεταξύ ακρίβειας και μνήμης.

Προφανώς:

- Ένας **μεγαλύτερος** πίνακας παρέχει πληρέστερη εικόνα του ιστορικού πρόσβασης ενός αντικειμένου αλλά αυξάνει τις απαιτήσεις μνήμης.
- Ένας **μικρότερος** πίνακας εστιάζει στις πιο πρόσφατες προσπελάσεις, μειώνοντας το απαιτούμενο overhead στη μνήμη.

Σε κάθε περίπτωση, ο αλγόριθμος αποδίδει μεγαλύτερη βαρύτητα στις πρόσφατες χρονικές στιγμές, καθώς αυτές είναι συνήθως πιο αντιπροσωπευτικές για τη μελλοντική χρησιμότητα του αντικειμένου.

Για τη μείωση της κατανάλωσης μνήμης, αντί να αποθηκεύουμε timestamps με ακρίβεια χιλιοστών του δευτερολέπτου (milliseconds) που καταλαμβάνουν 64 bits (8 bytes), χρησιμοποιούμε μόνο **24 bits (3 bytes)**. Αυτό είναι εφικτό, καθώς τα πρώτα ψηφία του timestamp (που αντιστοιχούν στην απόλυτη χρονική στιγμή) παραμένουν σταθερά λόγω της χρονικής εγγύτητας των δεδομένων και δε χρειάζεται να αποθηκεύονται. Αντίθετα, τα τελευταία ψηφία είναι αυτά που καθορίζουν τη σχετική διαφορά μεταξύ των timestamps.

Έτσι, η κάθε θέση του πίνακα καταλαμβάνει 3 bytes. Συνεπώς, ένας πίνακας 5 θέσεων, θα καταλάμβανε συνολικά 15 bytes.

3.1.2.1 Εισαγωγή Χρονικών Στιγμών και Κυκλική Μετατόπιση στον Πίνακα

Έστω ένας πίνακας «T» 5 θέσεων:

- Αρχική κατάσταση:
T: [_, _, _, _, _]
- Μετά την εισαγωγή των χρονικών στιγμών 1000, 2000, 3000, 4000, 5000:
T: [1000, 2000, 3000, 4000, 5000]
- Μετά την εισαγωγή της 6^{ης} χρονικής στιγμής 6000:
T: [2000, 3000, 4000, 5000, 6000]

3.1.3 Μετρητής για την καταγραφή της Συχνότητας Πρόσβασης σε δεδομένα

Εκτός από τον δυναμικό πίνακα, ο αλγόριθμος εμπλουτίζεται με έναν **μετρητή συχνότητας** που αυξάνεται κάθε φορά που ζητείται ένα αντικείμενο. Αυτός ο μετρητής καταγράφει το συνολικό αριθμό των προσπελάσεων, ανεξάρτητα από τις κυκλικές μετατοπίσεις που μπορεί να συμβούν στον πίνακα των timestamps. Με αυτόν τον τρόπο, εξασφαλίζεται η διατήρηση μιας ακριβούς μέτρησης της συχνότητας πρόσβασης για κάθε αντικείμενο, αποτρέποντας την πιθανότητα να διαγραφεί ένα συχνά χρησιμοποιούμενο δεδομένο κατά την αντικατάσταση στη μνήμη.

Ο μετρητής αυτός έχει μέγεθος **8 bits (1 byte)**, καθώς αυτός ο χώρος είναι επαρκής για την καταγραφή της συχνότητας πρόσβασης, χωρίς να απαιτείται η χρήση τύπων δεδομένων μεγαλύτερου μεγέθους, όπως int ή long.

3.1.4 Μετρικές mean και standard deviation

Ο υπολογισμός των μετρικών του μέσου όρου (*mean*) και της τυπικής απόκλισης (*standard deviation*), ενεργοποιείται κάθε φορά που η μνήμη ξεπεράσει το **100%** της χωρητικότητάς της. Η διαδικασία εκτελείται μόνο για τα τυχαία αντικείμενα (*candidates*) που επιλέγονται για αντικατάσταση και βασίζεται στις ηλικίες (*ages*) των χρονικών στιγμών πρόσβασης των αντικειμένων στη μνήμη.

Ο σκοπός που υπολογίζονται οι δύο αυτές μετρικές είναι για να φτιαχτούν, όπως θα δούμε στην επόμενη ενότητα, τα δυναμικά παράθυρα ηλικίας, για να σχεδιαστεί εν τέλει για κάθε αντικείμενο ένα ιστόγραμμα προσβάσεων.

Διαδικασία Υπολογισμού

Το πρόγραμμα διατρέχει όλες τις χρονικές στιγμές πρόσβασής του και υπολογίζει την ηλικία του κάθε γεγονότος ως:

$$age = now - AT$$

Formula 1: Definition of age

όπου *now* είναι ο τρέχων χρόνος και *AT* η χρονική στιγμή πρόσβασης

- Ενημερώνεται η ελάχιστη (*global min*) και μέγιστη (*global max*) ηλικία του μεταξύ όλων των υπό εξέταση αντικειμένων.
- Υπολογίζεται το συνολικό πλήθος των χρονικών στιγμών (*count*).
- Οι τιμές των ηλικιών αθροίζονται για τον υπολογισμό του μέσου όρου.
- Τα τετράγωνα των ηλικιών αθροίζονται για τον υπολογισμό της διακύμανσης.

Υπολογισμός Τελικών Τιμών

Μετά την επεξεργασία όλων των χρονικών στιγμών των επιλεγμένων αντικειμένων, οι μετρικές του μέσου όρου και της τυπικής απόκλισης υπολογίζονται ως εξής:

$$mean = \frac{\sum age}{count}$$

Formula 2: Mean Calculation

$$stddev = \sqrt{\frac{\sum age^2}{count} - mean^2}$$

Formula 3: Standard Deviation Calculation

3.1.5 Υπολογισμός παραθύρων ηλικίας

Τα παράθυρα χρονικών στιγμών πρόσβασης καθορίζονται βάσει του μέσου όρου και της τυπικής απόκλισης των ηλικιών, καθώς και τους $global\ min$ και $global\ max$ χρόνους πρόσβασης. Αυτά τα παράθυρα είναι χρήσιμα για την κατηγοριοποίηση των αντικειμένων ανάλογα με την πρόσφατη δραστηριότητά τους, βοηθώντας στην καλύτερη εκτίμηση της χρησιμότητάς τους για μελλοντική χρήση. Τα όρια των παραθύρων ορίζονται ως εξής:

- 1^ο παράθυρο: $[global_{min}, \max(global_{min}, mean - stddev))$
- 2^ο παράθυρο: $[\max(global_{min}, mean - stddev), mean]$
- 3^ο παράθυρο: $[mean, \min(global_{max}, mean + stddev))$
- 4^ο παράθυρο: $[\min(global_{max}, mean + stddev), global_{max}]$

Τα παράθυρα αυτά είναι *δυναμικά*, καθώς διαμορφώνονται κάθε φορά κατά τη διαδικασία αντικατάστασης. Η χρήση των μαθηματικών εννοιών του μέσου όρου και της τυπικής απόκλισης, εξασφαλίζουν ότι τα παράθυρα θα φτιαχτούν με σωστό τρόπο και για πιο πυκνές και για πιο αραιές τιμές.

Η χρήση των συναρτήσεων min και max εξασφαλίζει ότι τα όρια των παραθύρων παραμένουν εντός των φυσικών ορίων, αποτρέποντας την υπέρβαση των ελαχίστων και μεγίστων τιμών για την ακριβή κατηγοριοποίηση των δεδομένων.

3.1.5.1 Dense Παράδειγμα Υπολογισμού Παραθύρων Ηλικίας

Δεδομένα

- Αντικείμενο A_1
 - Age of access times: [1, 2, 3, 7]
- Αντικείμενο A_2
 - Age of access times: [1, 2, 3, 5]

Υπολογισμοί

- $global_{min} = \min(\min_{A_1}(1, 2, 3, 7), \min_{A_2}(1, 2, 3, 5)) = 1$
- $global_{max} = \min(\min_{A_1}(1, 2, 3, 7), \min_{A_2}(1, 2, 3, 5)) = 7$
- $mean = \frac{(1+2+3+7+1+2+3+5)}{8} = 3$
- $sum_{ages_{squared}} = 1^2 + 2^2 + 3^2 + 7^2 + 1^2 + 2^2 + 3^2 + 5^2 = 102$

- $sttdev = \sqrt{\frac{102}{8} - 3^2} = \sqrt{12.75 - 9} \approx 1.93$
- $mean - sttdev = 3 - 1.93 = 1.07$
- $mean + sttdev = 3 + 1.93 = 4.93$

Παράθυρα

- 1^ο παράθυρο: [1, 1.07)
- 2^ο παράθυρο: [1.07, 3)
- 3^ο παράθυρο: [3, 4.93)
- 4^ο παράθυρο: [4.93, 7]

3.1.5.2 Sparse Παράδειγμα Υπολογισμού Παραθύρων Ηλικίας

Δεδομένα

- Αντικείμενο **A₁**
 - Age of access times: [1, 20, 30, 100]
- Αντικείμενο **A₂**
 - Age of access times: [1, 20, 30, 80]

Υπολογισμοί

- $global_{min} = \min(\min_{A_1}(1, 20, 30, 100), \min_{A_2}(1, 20, 30, 80)) = 1$
- $global_{max} = \min(\min_{A_1}(1, 20, 30, 100), \min_{A_2}(1, 20, 30, 80)) = 100$
- $mean = \frac{(1+20+30+100+1+20+30+80)}{8} = 35.25$
- $sttdev = \sqrt{\frac{(1-35.25)^2 + (20-35.25)^2 + \dots + (30-35.25)^2 + (80-35.25)^2}{8}} = \sqrt{\frac{9061.5}{8}} \approx 33.65$
- $mean - sttdev = 35.25 - 33.65 = 1.6$
- $mean + sttdev = 35.25 + 33.65 = 68.9$

Παράθυρα

- 1^ο παράθυρο: [1, 1.6)
- 2^ο παράθυρο: [1.6, 35.25)
- 3^ο παράθυρο: [35.25, 68.9)
- 4^ο παράθυρο: [68.9, 100]

3.1.6 Υπολογισμός τελικού Score

Η διαδικασία υπολογισμού του score για κάθε αντικείμενο βασίζεται στη δημιουργία ενός ιστογράμματος, που βασίζεται στη δυναμική κατανομή των προσβάσεων σε διαφορετικά χρονικά παράθυρα, με κάθε παράθυρο να έχει παραμετροποιήσιμο βάρος, το οποίο καθορίζει τη συμβολή του στο συνολικό score.

Γίνεται χρήση ενός παραμετροποιήσιμου πίνακα *WINDOW_WEIGHTS*, που περιλαμβάνει τα βάρη για κάθε παράθυρο, και καθορίζει τη συνεισφορά του κάθε παραθύρου στο τελικό score. Σε κάθε παράθυρο αντιστοιχεί και το αντίστοιχο βάρος, με τα βάρη να μειώνονται όσο αυξάνεται η ηλικία των προσβάσεων (δηλαδή, τα πιο πρόσφατα παράθυρα έχουν μεγαλύτερο βάρος σε σχέση με τα παλαιότερα).

Για κάθε πρόσβαση, υπολογίζεται η ηλικία της και, ανάλογα με αυτήν, κατατάσσεται στο παράθυρο. Αντίστοιχα, ο αντίστοιχος μετρητής αυτού του παραθύρου (*window_counts*) αυξάνεται.

Τέλος, προστίθεται ένας μικρός παράγοντας για την εκκαθάριση ισοψηφιών (*tie_breaker*), ο οποίος βασίζεται στον συνολικό αριθμό προσβάσεων του αντικειμένου.

Ο τελικός υπολογισμός του score πραγματοποιείται μέσω της παρακάτω σχέσης:

$$score = \sum_{i=1}^{NUM_WINDOWS} (window_counts[i] \times WINDOW_WEIGHTS[i]) + tie_breaker$$

Formula 4: Total (Eviction) Score Calculation

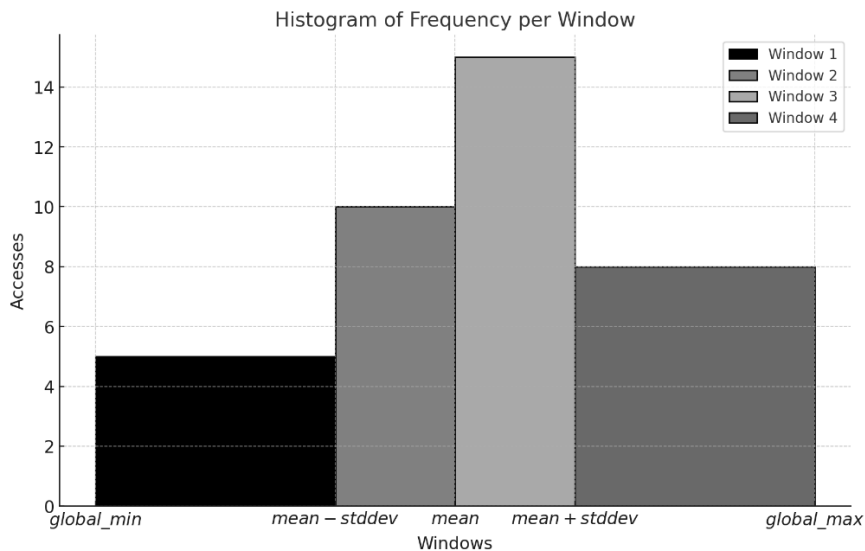


Figure 3: Ιστόγραμμα.

3.1.6.1 Παράδειγμα Υπολογισμού Τελικού Score

Δεδομένα

- $WINDOW_WEIGHTS = \{0.4, 0.3, 0.2, 0.15\}$
- $NOW = 10000$
- Αντικείμενο A_1
 - Access Times: 9000, 3000, 2000
 - Frequency: 3
- Αντικείμενο A_2
 - Access Times: 1500, 2000, 3000
 - Frequency: 3

Υπολογισμοί

- $ages_{A_1} = [1000, 7000, 9000]$
- $ages_{A_2} = [8500, 8000, 7000]$
- $global_{min} = \min(\min_{A_1}(1000, 7000, 9000), \min_{A_2}(8500, 8000, 7000)) = 1000$
- $global_{max} = \min(\min_{A_1}(1000, 7000, 9000), \min_{A_2}(8500, 8000, 7000)) = 9000$
- $sum_{ages} = 1000 + 7000 + 8000 + 8500 + 8000 + 7000 = 39500$
- $mean = \frac{39500}{6} \cong 6583$
- $sum_{ages_{squared}} = 1000^2 + 7000^2 + 8000^2 + 8500^2 + 8000^2 + 7000^2 =$
 $1000000 + 49000000 + 64000000 + 72250000 + 64000000 + 49000000 =$
 316250000
- $sttdev = \sqrt{\frac{316250000}{6} - \left(\frac{39500}{6}\right)^2} = \sqrt{52708333.33 - 43383333.33} \approx 3054$
- $windows = [1000, 3529, 6583, 9637]$
- $score_{A_1} = (1 \times 0.4) + (0 \times 0.3) + (2 \times 0.2) + (0 \times 0.15) + 0.01 \times 3 = 0.4 +$
 $0 + 0.4 + 0 + 0.03 = 0.83$
- $score_{A_2} = (0 \times 0.4) + (0 \times 0.3) + (2 \times 0.2) + (1 \times 0.15) + 0.01 \times 3 = 0 + 0 +$
 $0.4 + 0.15 + 0.03 = 0.58$

Συμπέρασμα

Το αντικείμενο A_1 έχει το υψηλότερο score, και θεωρείται πιο σημαντικό σε σχέση με το A_2 , Αυτό οφείλεται στο γεγονός ότι η πρόσφατη πρόσβαση στο A_1 , επηρέασε θετικά το score του.

3.2 Περιβάλλον και Παραμετροποίηση

Για την ανάπτυξη και αξιολόγηση του συστήματος, διαμορφώθηκε ένα περιβάλλον εκτέλεσης βασισμένο σε εικονική μηχανή (VM), όπου εγκαταστάθηκαν και ρυθμίστηκαν οι απαραίτητες τεχνολογίες και εργαλεία. Το περιβάλλον αυτό εξασφαλίζει την απομόνωση των υπηρεσιών και την ορθή παρακολούθηση των πειραμάτων, επιτρέποντας την, κατά δύναμη, ακριβή και επαναλαμβανόμενη μέτρηση των επιδόσεων.

3.2.1 Υποδομή και Περιβάλλον Εργασίας

Η υλοποίηση πραγματοποιήθηκε σε μια εικονική μηχανή (VM) μέσω του VirtualBoxVM με λειτουργικό σύστημα Ubuntu (64-bit), και 5 CPU cores και 13.2 GB RAM.

Δημιουργήθηκαν δύο projects, το *Redis Classic* και το *Redis Optis*, τα οποία αντιστοιχούν στην κανονική υλοποίηση της Redis και στην νέα υλοποίηση που παρουσιάζουμε στην παρούσα εργασία.

Η Redis version που χρησιμοποιήθηκε και στις δύο περιπτώσεις είναι η release version **7.2.x**, η οποία περιλαμβάνει τις αλλαγές που είχαν γίνει στη Redis 3.0 και χρησιμοποιεί τον Approximated LRU.

3.2.2 Υλοποίηση και Επικοινωνία

Η υλοποίηση βασίζεται στο κλασικό μοντέλο Πελάτη - Εξυπηρετητή (Client - Server). Στον ρόλο του εξυπηρετητή είναι το Redis instance, όπου η υλοποίηση έγινε σε γλώσσα προγραμματισμού C. Στον ρόλο του πελάτη αναπτύχθηκε ένα νέο σύστημα σε γλώσσα προγραμματισμού **Java** χρησιμοποιώντας τις εξής τεχνολογίες:

- **RESTful API:** Η επικοινωνία με την Redis έγινε με τη δημιουργία ενός client συστήματος και μέσω REST με τη δημιουργία ενός RESTful API
- **Java 11:** Το client σύστημα υλοποιήθηκε με Java 11.
- **Spring Boot:** Το client σύστημα υλοποιήθηκε με τη βοήθεια του Spring Boot 2.7.11.
- **Micrometer:** Έγινε χρήση της βιβλιοθήκης Micrometer, η οποία επιτρέπει την εξαγωγή στατιστικών σε διάφορες μορφές και την ενσωμάτωσή τους σε εργαλεία παρακολούθησης, όπως το Prometheus.

- **Postgres:** Για την αποθήκευση των δεδομένων, χρησιμοποιήθηκε η *Postgres (postgres:14.2-alpine)* και συγκεκριμένα με δύο instances της να σηκώνονται μέσω Docker, ένα για την *Redis Classic* και ένα για την *Redis Optis*.
- Η σύνδεση με την εφαρμογή έγινε μέσω *JDBC*, με τη χρήση του *JPA*.

Το API βασίζεται σε HTTP requests, υποστηρίζοντας CRUD για τη διαχείριση δεδομένων, όπως:

POST https://{{url}}/object

GET https://{{url}}/object/{{id}}

3.2.3 Εργαλεία Πειραμάτων και Παρακολούθησης

Για την εκτέλεση των πειραμάτων και τη συλλογή δεδομένων, χρησιμοποιήθηκαν τα ακόλουθα εργαλεία:

- **Prometheus** (Dockerized): Χρησιμοποιήθηκε για τη συλλογή και αποθήκευση των μετρικών που παράγονται από το σύστημα.
Docker Image: prom/prometheus:v2.39.0
- **Redis Exporter** (Dockerized): Χρησιμοποιήθηκε για τη συλλογή μετρικών, σχετικά με τη μνήμη και τη CPU της Redis.
Docker Image: image oliver006/redis_exporter:v1.62.0
- **Grafana** (Dockerized): Χρησιμοποιήθηκε για την οπτικοποίηση των μετρικών, παρέχοντας δυναμικά διαγράμματα και dashboards για την ανάλυση των δεδομένων.
Docker Image: Grafana/Grafana:9.5.0
- **JMeter**: Χρησιμοποιήθηκε για την προσομοίωση φόρτου στο σύστημα, καθώς και για τη δημιουργία και εκτέλεση των διαφορετικών πειραμάτων.
Version: 5.6.3

Κεφάλαιο 4: Πειραματική Αξιολόγηση

Σε αυτό το κεφάλαιο παρουσιάζεται η πειραματική αξιολόγηση της προτεινόμενης μεθόδου αντικατάστασης δεδομένων HLRU (*Redis Optis*), συγκριτικά με τον παραδοσιακό αλγόριθμο LRU (*Redis Classic*).

Για την εκτέλεση, λοιπόν, της πρώτης κατηγορίας πειραμάτων, διαμορφώθηκαν κατάλληλα οι πειραματικές ρυθμίσεις, ώστε να αποτυπωθεί με ακρίβεια η συμπεριφορά των αλγορίθμων σε διαφορετικά σενάρια. Η διαμόρφωση κάθε πειράματος περιλαμβάνει τα παρακάτω βήματα και οποιαδήποτε διαφοροποίηση θα αναφέρεται στις παραμέτρους το εκάστοτε πειράματος:

- **Αρχικοποίηση της cache:** Φόρτωση της cache με δεδομένα μέχρι το μέγιστο επιτρεπτό όριο μνήμης *maxmemory*.
- **Διαμόρφωση Μοτίβου Δεδομένων:** Ανάκτηση των δεδομένων, ανάλογα με το μοτίβο πρόσβασης του πειράματος.
- **Δημιουργία συνθηκών αντικατάστασης:** Προστίθεται επιπλέον όγκος δεδομένων, ίσος με το 1/3 του συνολικού μεγέθους των αρχικών δεδομένων, ώστε να προκληθούν αντικαταστάσεις αντικειμένων στη μνήμη.
- **Ανάκτηση αντικειμένων βάσει Μοτίβου:** Εκτελείται ανάκτηση αντικειμένων ανάλογα με το μοτίβο πρόσβασης του πειράματος, ίση με το 1/3 του συνολικού μεγέθους των δεδομένων.
- **Cache Miss Penalty:** Για όλα τα πειράματα, το penalty για κάθε cache miss έχει οριστεί στα 300ms, τιμή που αντιστοιχεί στον μέσο όρο καθυστέρησης πρόσβασης σε ρεαλιστικά συστήματα μνήμης. Η τιμή αυτή είναι παραμετροποιήσιμη.
- **Παράμετρος *maxmemory-samples*:** Η παράμετρος που ορίζει πόσα αντικείμενα θα επιλεγούν κατά τη δειγματοληψία είναι η default τιμή 5.
- **Παράμετρος *WINDOW_WEIGHTS*:** Σε όλα τα πειράματα χρησιμοποιήθηκαν τα βάρη {0.4, 0.3, 0.2, 0.15}.

Η δεύτερη κατηγορία πειραμάτων εξετάζει τη συμπεριφορά του **HLRU** υπό διαφορετικές ρυθμίσεις της παραμέτρου **AT (Access Time)**, η οποία καθορίζει το πλήθος των θέσεων που διατηρούνται στον πίνακα προσβάσεων του αντικειμένου. Παράλληλα, διεξήχθησαν θεωρητικοί υπολογισμοί για τη χρήση μνήμης του HLRU σε σχέση με το AT, προκειμένου να εκτιμηθεί η βέλτιστη επιλογή της παραμέτρου ανάλογα με διάφορες συνθήκες.

Τέλος, για κάθε σενάριο αποτυπώνονται τα βήματα που ακολουθήθηκαν συνοδευόμενα με στιγμιότυπα στα οποία φαίνονται οι παρακάτω μετρικές:

- Rate ($\frac{\text{Συνολικός Χρόνος Ανάκτησης}}{\text{Αριθμός Ανακτήσεων}}$)
- Used Memory vs Max Memory
- CPU Usage: Ποσοστό απασχόλησης της CPU
- Cache Miss Count (Μετρητής Αποτυχιών Εύρεσης Αντικειμένου στην Cache): Αυτή η μετρική καταγράφει πόσες φορές η αναζήτηση ενός αντικειμένου στη μνήμη cache απέτυχε, δηλαδή απαιτήθηκε ανάκτηση του αντικειμένου από το κύριο αποθηκευτικό σύστημα. Ο αριθμός των αποτυχιών cache (cache misses) είναι ένας από τους πιο κρίσιμους δείκτες απόδοσης για τους αλγορίθμους αντικατάστασης, καθώς αντανakλά άμεσα την αποτελεσματικότητα της μνήμης cache και την ανάγκη για αντικαταστάσεις.

Από όλες τις μετρικές που καταγράφηκαν, το **Cache Miss Count** είναι η πλέον καθοριστική, καθώς αποδίδει με απόλυτο τρόπο την απόδοση του αλγορίθμου, σε αντίθεση με άλλες μετρικές, όπως το Rate, που παρέχει σχετικές και έμμεσες εκτιμήσεις. Το Cache Miss Count δείχνει άμεσα πόσες φορές η αναζήτηση δεδομένων στη μνήμη cache απέτυχε, οδηγώντας σε αργή ανάκτηση από το κύριο αποθηκευτικό σύστημα. Η καθυστέρηση αυτή εξαρτάται από πολλές παραμέτρους, όπως είναι η τοπολογία του συστήματος, η πολυπλοκότητα της δομής που ανακτάται. Επομένως, αποτελεί τον πιο αξιόπιστο δείκτη για την αξιολόγηση της αποτελεσματικότητας των αλγορίθμων αντικατάστασης.

4.1 Random Σενάριο (LRU vs HLRU)

Στο συγκεκριμένο σενάριο, η εμφάνιση ή μη-εμφάνιση ενός αντικειμένου καθορίζεται αποκλειστικά από έναν τυχαίο μηχανισμό, εξασφαλίζοντας τη μη ύπαρξη προκαθορισμένων μοτίβων πρόσβασης.

Διαμόρφωση σεναρίου:

- **HLRU με $AT = 1$**
- **Ανάκτηση αντικειμένων βάσει Μοτίβου:** Εκτελείται τυχαία ανάκτηση αντικειμένων.

Για τη διασφάλιση της συγκρισιμότητας μεταξύ των υλοποιήσεων, χρησιμοποιήθηκε γεννήτρια τυχαίων αριθμών. Σε κάθε εκτέλεση του πειράματος, παράγεται ένα διαφορετικό σύνολο τυχαίων τιμών, το οποίο χρησιμοποιείται και στις 2 υλοποιήσεις. Έτσι, και οι 2 υλοποιήσεις υποβάλλονται σε ταυτόσημες συνθήκες προσπέλασης και αντικατάστασης εντός του ίδιου σεναρίου.

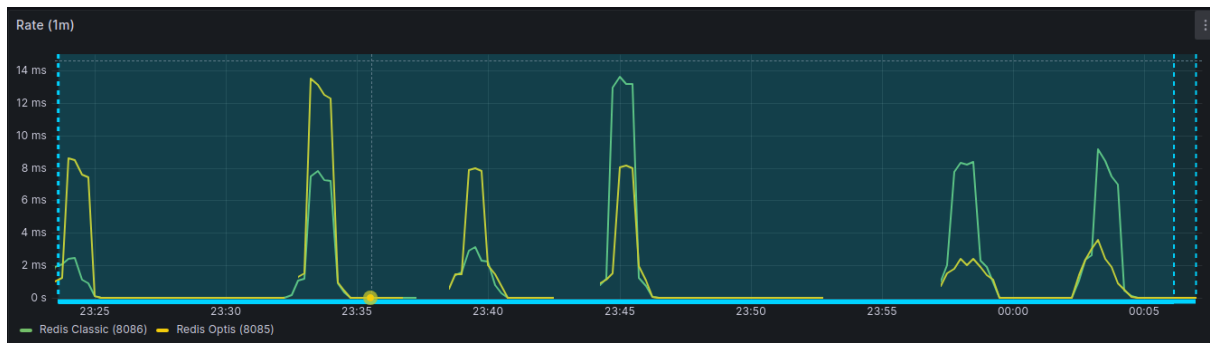


Figure 4: Random Scenario - Rate



Figure 5: Random Scenario - Used Memory vs Max Memory

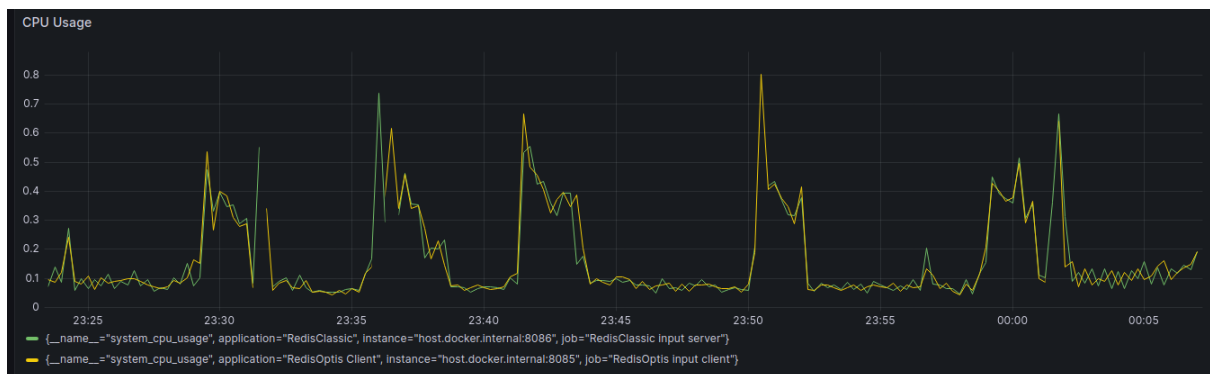


Figure 6: Random Scenario - CPU Usage

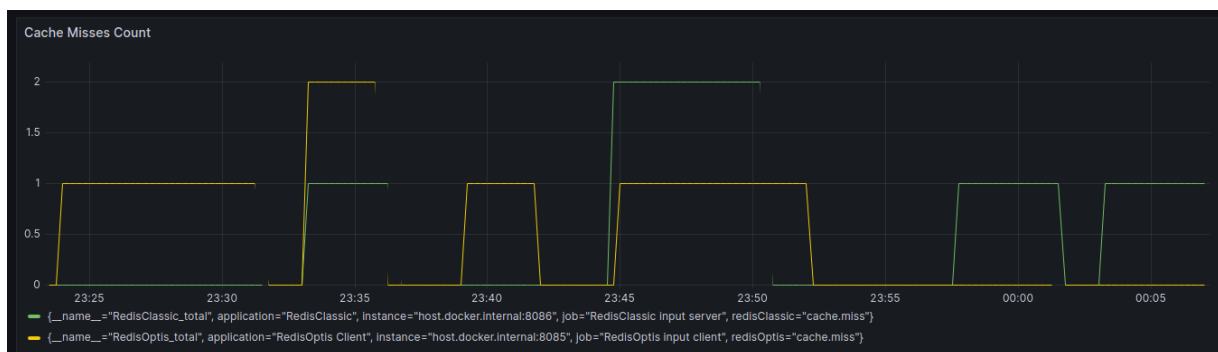


Figure 7: Random Scenario - Cache Miss Count

4.2 Σενάριο Ευνοϊκό σε LRU με $f > 1$ (LRU vs HLRU)

Το συγκεκριμένο σενάριο έχει σχεδιαστεί με τρόπο που ευνοεί τον αλγόριθμο LRU. Συγκεκριμένα, στη διαδικασία διαμόρφωσης του μοτίβου δεδομένων αυξάνεται τεχνητά η συχνότητα πρόσβασης σε συγκεκριμένα αντικείμενα. Κατά την ανάκτηση, όμως, αυτά δε ζητούνται, και άρα η δημοφιλία τους, δεν αντικατοπτρίζεται άμεσα στο μοτίβο αναζήτησης.

Αποτελεί μια πολύ συγκεκριμένη περίπτωση, όπου η συχνότητα των προσβάσεων δρα παραπλανητικά για τον αλγόριθμο HLRU, καθώς δεν τον καθοδηγεί σωστά στην επιλογή αντικειμένων προς αντικατάσταση. Ο μηχανισμός βαρών που χρησιμοποιεί το HLRU επηρεάζεται από τη συνολική συχνότητα πρόσβασης, γεγονός που τον οδηγεί σε λιγότερο αποδοτικές αποφάσεις συγκριτικά με τον LRU. Ως αποτέλεσμα, το HLRU δυσκολεύεται να αποδώσει καλύτερα από τον LRU, καθώς η στρατηγική του δεν συνάδει με το συγκεκριμένο μοτίβο προσβάσεων.

Διαμόρφωση σεναρίου:

- **HLRU με $AT = 10$**
- **Διαμόρφωση Μοτίβου Δεδομένων:** Δεδομένα με συχνότητα $f > 1$,
- **Ανάκτηση αντικειμένων:** Εκτελείται ανάκτηση αντικειμένων με ευνοϊκό τρόπο για την LRU, ίση με το $1/3$ του συνολικού μεγέθους των δεδομένων.

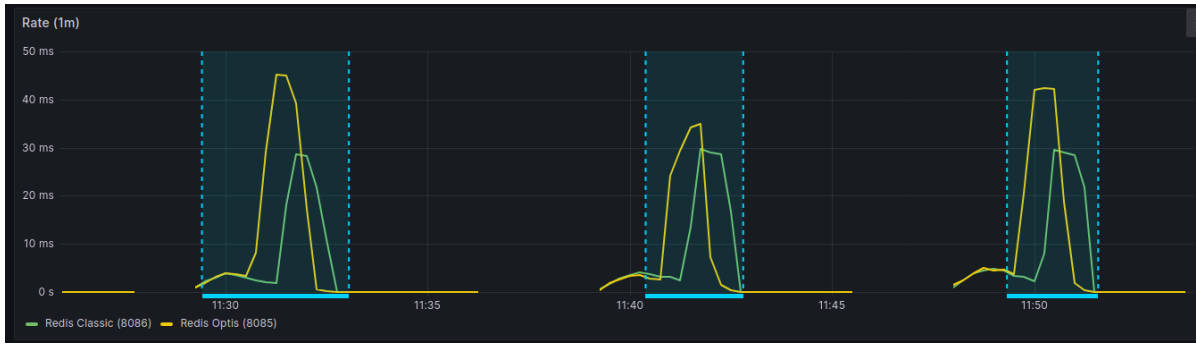


Figure 8: LRU-Favoured Scenario - Rate

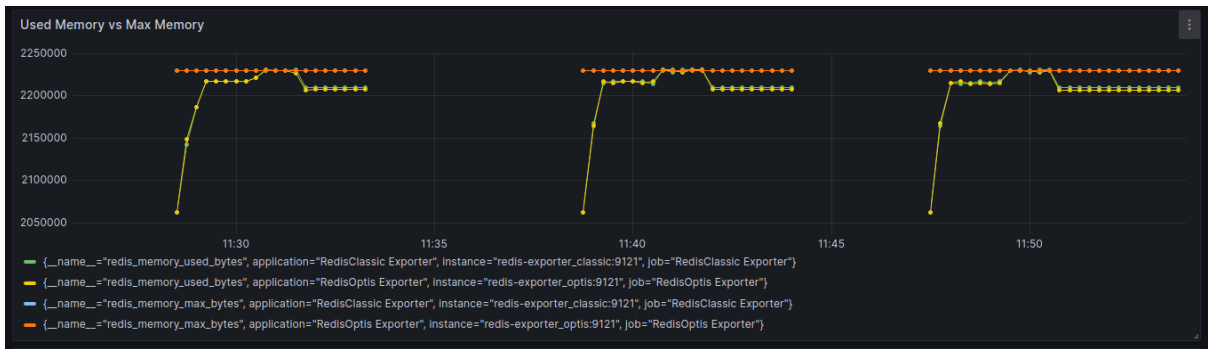


Figure 9: LRU-Favoured Scenario - Used Memory vs Max Memory

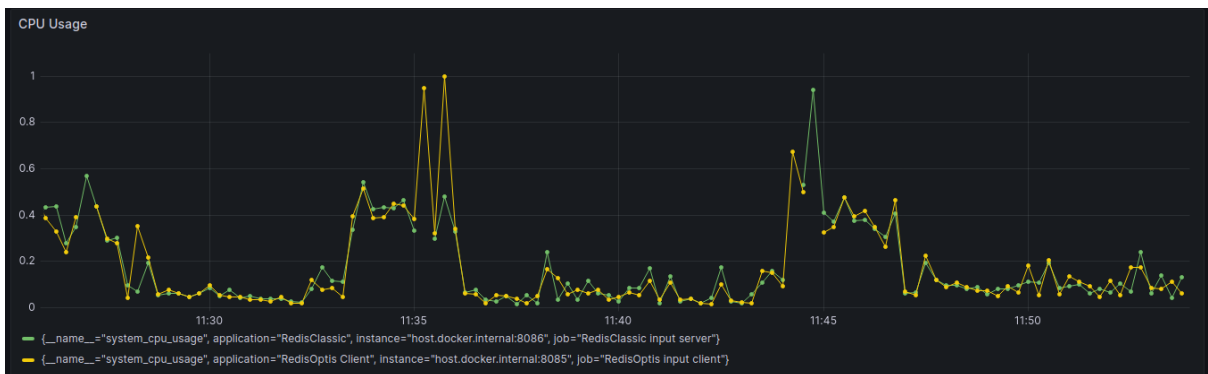


Figure 10: LRU-Favoured Scenario - CPU Usage



Figure 11: LRU-Favoured Scenario - Cache Miss Count

4.3 Σενάριο LRU με $f = 1$ (LRU vs HLRU)

Σε σχέση με το προηγούμενο σενάριο, εδώ όλα τα αντικείμενα καλούνται μία φορά κατά τη φάση της ανάκτησης δεδομένων ($f = 1$), οπότε η παράμετρος της συχνότητας δεν αποτελεί κριτήριο επιλογής ή μη επιλογής ενός αντικειμένου ως πιο σημαντικό.

Διαμόρφωση σεναρίου:

- **HLRU με $AT = 10$**
- **Διαμόρφωση Μοτίβου Δεδομένων:** Δεδομένα με συχνότητα $f = 1$,
- **Ανάκτηση αντικειμένων:** Εκτελείται ανάκτηση αντικειμένων με ευνοϊκό τρόπο για την LRU, ίση με το $1/3$ του συνολικού μεγέθους των δεδομένων.

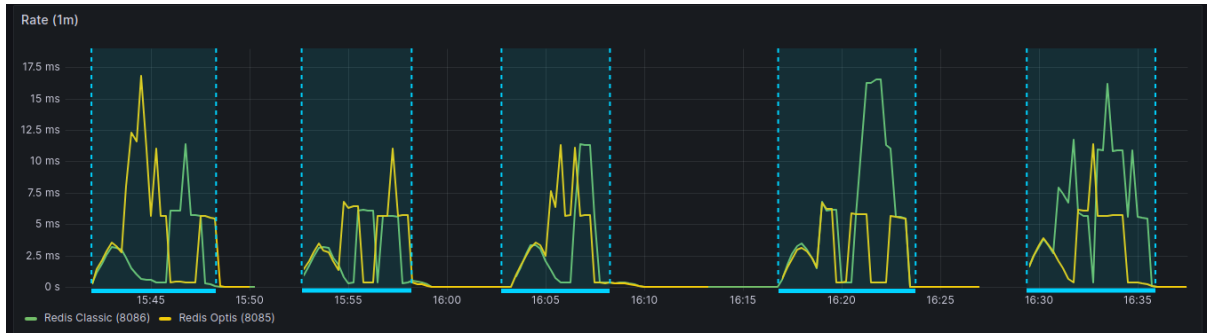


Figure 12: LRU ($F = 1$) - Rate



Figure 13: LRU ($F = 1$) - Used Memory vs Max Memory

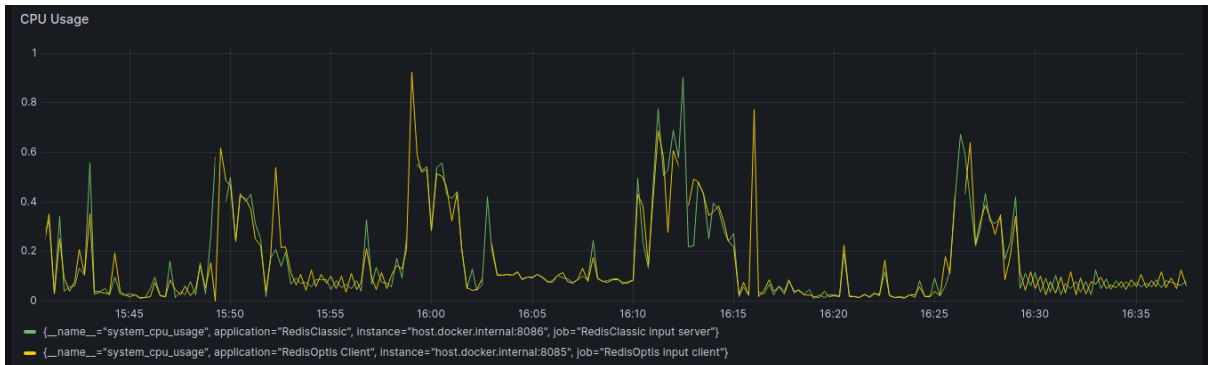


Figure 14: LRU ($F = 1$) - CPU Usage

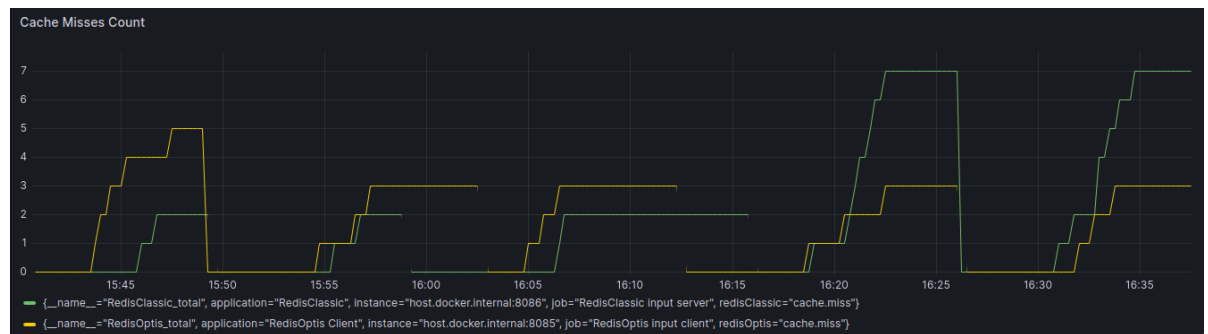


Figure 15: LRU ($F = 1$) - Cache Miss Count

4.4 Σενάριο Hot & Cold (LRU vs HLRU)

Το σενάριο αυτό έχει σχεδιαστεί ώστε να προσομοιώνει μια κατάσταση στην οποία υπάρχει ένα σύνολο αντικειμένων που είναι ιδιαίτερα δημοφιλή (*Hot*) και ένα άλλο σύνολο που έχει πολύ χαμηλή συχνότητα προσβάσεων (*Cold*). Στόχος αυτού του πειράματος είναι να αξιολογηθεί η ικανότητα των αλγορίθμων αντικατάστασης να διατηρούν στη μνήμη τα *Hot* αντικείμενα και να απομακρύνουν τα *Cold*, διασφαλίζοντας έτσι υψηλότερη αποδοτικότητα στην αποθήκευση δεδομένων.

Διαμόρφωση σεναρίου:

- **HLRU με $AT = 5$**
- **Διαμόρφωση Μοτίβου Δεδομένων:** Υποσύνολο των δεδομένων θεωρείται ως *Hot*, με υψηλή συχνότητα προσβάσεων, ενώ το υπόλοιπο θεωρείται *Cold*, με πολύ σπάνιες ή ακόμα και μοναδικές προσβάσεις
- **Ανάκτηση αντικειμένων:** Εκτελείται ανάκτηση αντικειμένων με τρόπο που συνάδει με τη δημοφιλία εκάστου αντικειμένου, ίση με το $1/3$ του συνολικού μεγέθους των δεδομένων.

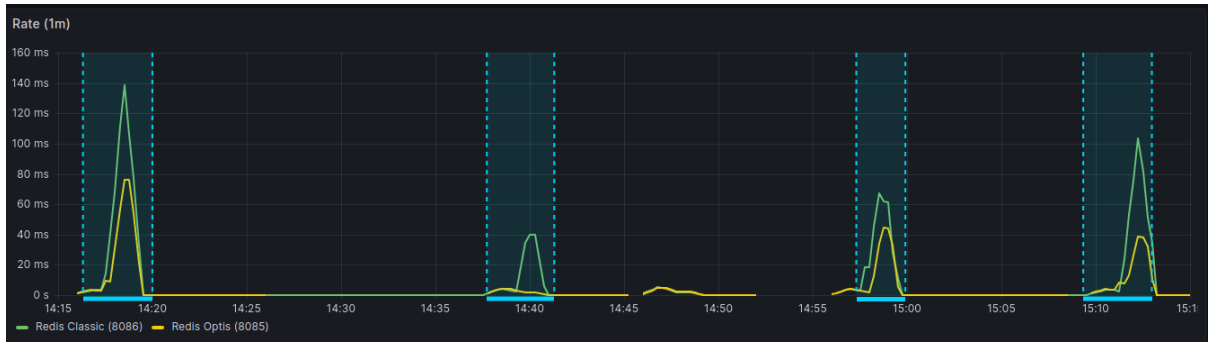


Figure 16: Hot & Cold - Rate

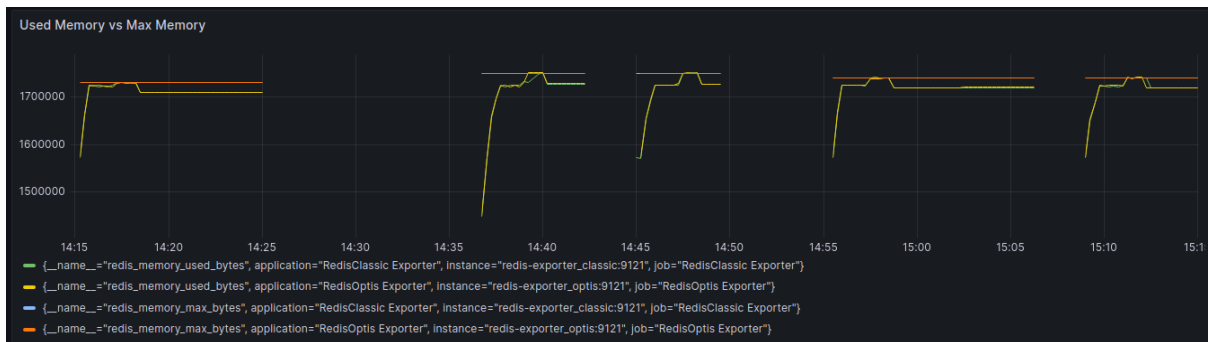


Figure 17: Hot & Cold - Used Memory vs Max Memory

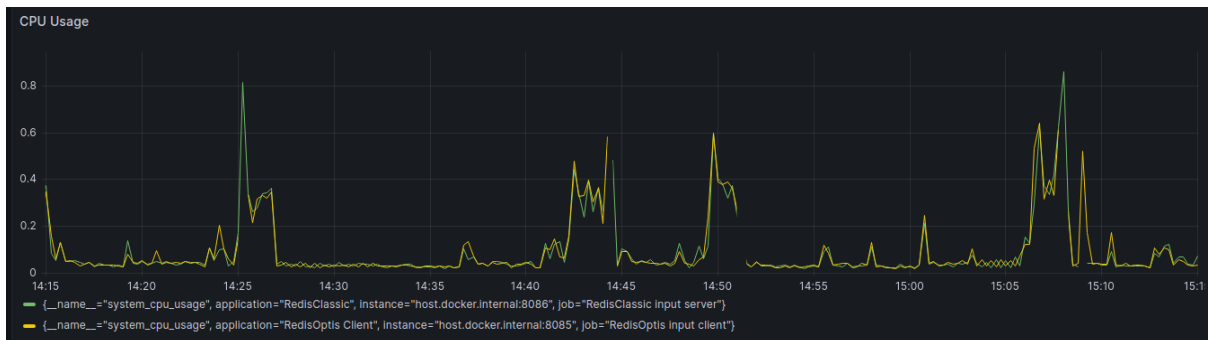


Figure 18: Hot & Cold - CPU Usage

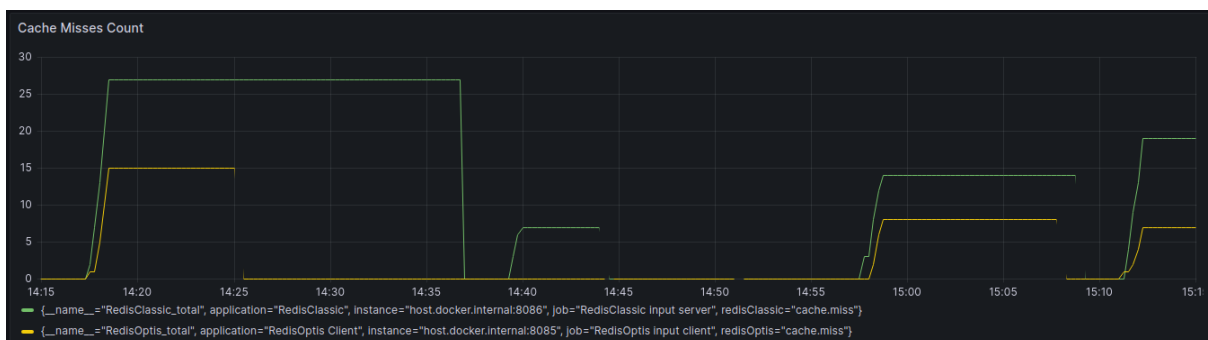


Figure 19: Hot & Cold - Cache Misses Count

4.5 Σενάριο Σειριακής Σάρωσης (LRU vs HLRU)

Το σενάριο αυτό έχει σχεδιαστεί ώστε να προσομοιώνει μια κατάσταση στην οποία η σημασία των αντικειμένων στη μνήμη διαμορφώνεται από μια συνδυασμένη στρατηγική πρόσφατης χρήσης και συχνότητας. Συγκεκριμένα, τα αντικείμενα που εισάγονται στη cache αποκτούν **διάφορες ιδιότητες**, όπως να είναι πιο πρόσφατα προσπελασμένα ή να έχουν μεγαλύτερη συχνότητα πρόσβασης. Στη συνέχεια, εκτελείται μία **σειριακή αναζήτηση** για την ανάκτηση δεδομένων.

Διαμόρφωση σεναρίου:

- **HLRU με $AT = 1$**
- **Διαμόρφωση Μοτίβου Δεδομένων:** Δεδομένα που είναι και recent αλλά και frequent αντικείμενα, αλλά και συνδυασμοί αυτών. Εκτελείται στο τέλος μια σειριακή αναζήτηση.

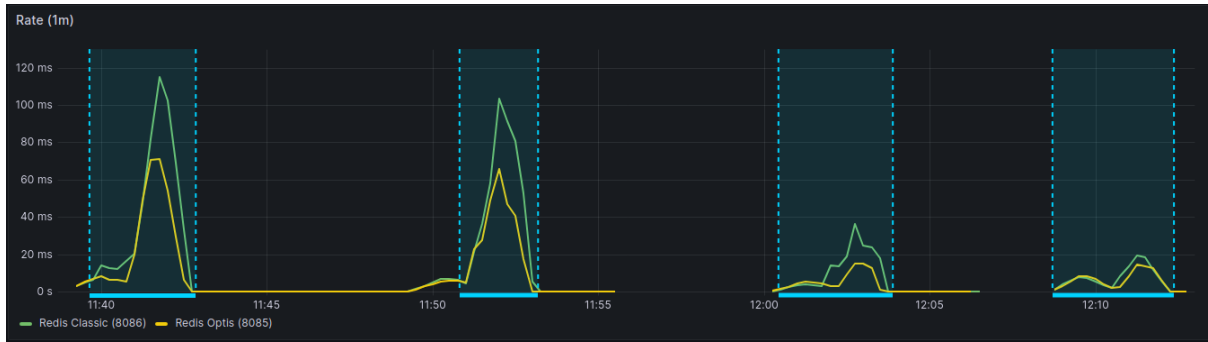


Figure 20: Scan Scenario - Rate



Figure 21: Scan Scenario - Used Memory vs Max Memory

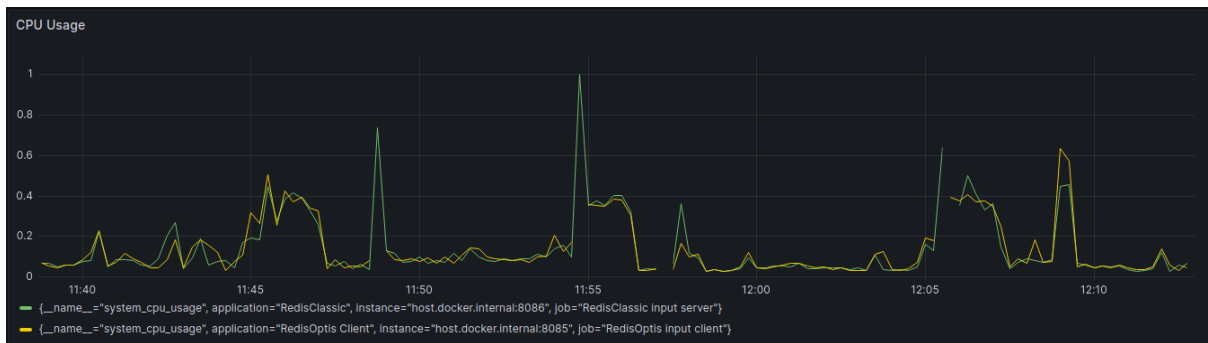


Figure 22: Scan Scenario - CPU Usage

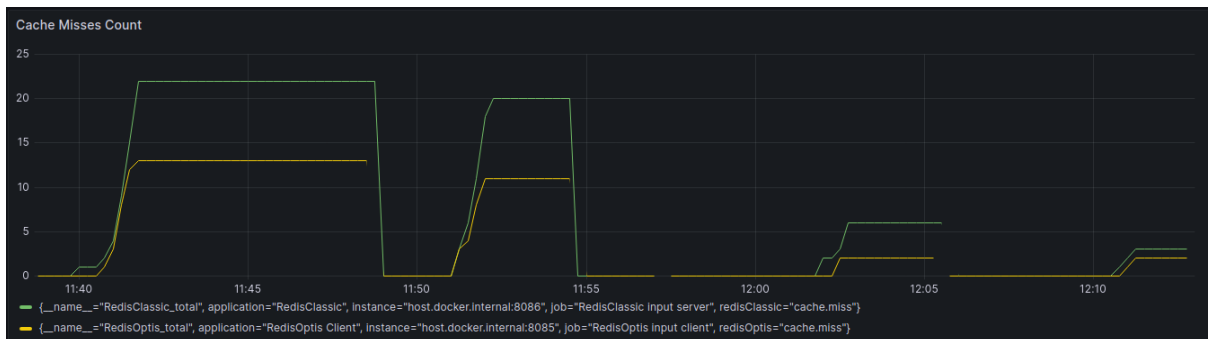


Figure 23: Scan Scenario - Cache Miss Count

4.6 Σενάρια Σύγκρισης HLRU ($AT = \{1, 2, 3, 4, 5\}$)

Σε αυτήν την ενότητα, θα διεξάγουμε πειράματα και συγκρίσεις πάνω στις διαφορετικές τιμές της παραμέτρου AT . Σκοπός δεν είναι η άμεση σύγκριση με την LRU, αλλά η παρακολούθηση της απόδοσης του αλγορίθμου με διαφορετικές τιμές AT (tuning). Στα σενάρια αυτά τα δεδομένα έχουν όλα συχνότητα $f = 5$, με κάποια αντικείμενα να καλούνται περισσότερες φορές πρόσφατα, ενώ άλλα να καλούνται λιγότερες φορές πρόσφατα.

Διαμόρφωση σεναρίου:

- **Διαμόρφωση Μοτίβου Δεδομένων:** Δεδομένα καλούνται και διαμορφώνονται βάσει του μοτίβου: 1 22 333 4444 5555 6 77 888 9999 ... 1111 222 33 4 6666 777 88 9 ...
- **Ανάκτηση αντικειμένων:** Εκτελείται ανάκτηση αντικειμένων με τρόπο που συνάδει με τη δημοφιλία εκάστου αντικειμένου, ίση με το $1/3$ του συνολικού μεγέθους των δεδομένων.

4.6.1 HLRU ($AT = \{1, 2, 3, 4, 5\}$) - Rate



Figure 24: HLRU ($AT = \{1, 2, 3, 4, 5\}$) - Rate

4.6.2 HLRU ($AT = \{1, 2, 3, 4, 5\}$) - Used Memory vs Max Memory



Figure 25: HLRU ($AT = \{1, 2, 3, 4, 5\}$) - Used Memory vs Max Memory

4.6.3 HLRU ($AT = \{1, 2, 3, 4, 5\}$) - CPU Usage



Figure 26: HLRU ($AT = \{1, 2, 3, 4, 5\}$) - CPU Usage

4.6.4 HLRU ($AT = \{1, 2, 3, 4, 5\}$) - Cache Miss



Figure 27: HLRU ($AT = \{1, 2, 3, 4, 5\}$) - Cache Miss

4.6.5 HLRU (AT = {1, 2, 3, 4, 5}) - Υπολογισμοί Μνήμης

Στην ενότητα αυτή θα ασχοληθούμε με τη μνήμη που χρειάζεται η υλοποίηση της HLRU ανάλογα με τις διάφορες τιμές του AT. Όπως αναλύσαμε στο προηγούμενο Κεφάλαιο, η HLRU απαιτεί:

- 1 byte παραπάνω για την παράμετρο της συχνότητας, και
- 3 bytes για κάθε μία θέση του δυναμικού πίνακα AT.

Επειδή ο αλγόριθμος LRU περιέχει ένα πεδίο *lru* το οποίο καταλαμβάνει 3 bytes, και αυτό δε χρησιμοποιείται στην υλοποίησή μας, αφαιρείται από τους υπολογισμούς.

Συνεπώς, για ένα αντικείμενο με AT = 1, η επιπλέον μνήμη που απαιτείται είναι μόνο 1 byte.

Παραδοχές και Δεδομένα

- Συνολική Διαθέσιμη Μνήμη: **2,048,000,000 bytes (2 GB)**
- **Μικρά** Αντικείμενα: **512 bytes (0.5 KB)**
- **Μεσαία** Αντικείμενα: **5,120 bytes (5 KB)**
- **Μεγάλα** Αντικείμενα: **5,000,000 bytes (5 MB)**

Μνήμη Ανά Αντικείμενο			
Τιμές AT	Μικρά Αντικείμενα	Μεσαία Αντικείμενα	Μεγάλα Αντικείμενα
AT = 1	512 + 1 = 513 bytes	5,120 + 1 = 5,121 bytes	5,000,000 + 1 = 5,000,001 bytes
AT = 2	512 + 1 + 3 = 516 bytes	5,120 + 1 + 3 = 5,124 bytes	5,000,000 + 1 + 3 = 5,000,004 bytes
AT = 3	512 + 1 + 6 = 519 bytes	5,120 + 1 + 6 = 5,127 bytes	5,000,000 + 1 + 6 = 5,000,007 bytes
AT = 4	512 + 1 + 9 = 522 bytes	5,120 + 1 + 9 = 5,130 bytes	5,000,000 + 1 + 9 = 5,000,010 bytes
AT = 5	512 + 1 + 12 = 525 bytes	5,120 + 1 + 12 = 5,133 bytes	5,000,000 + 1 + 12 = 5,000,013 bytes

Table 1: HLRU - Μνήμη ανά Κατηγορία Αντικειμένου.

Μνήμη για σταθερό αριθμό αντικειμένων ανάλογα με το μέγεθος			
Αντικείμενα που χωράνε σε 2 GB	Μικρά 4.000.000 αντικείμενα	Μεσαία 400.000 αντικείμενα	Μεγάλα 409 αντικείμενα
AT = 1	$4.000.000 \times 513 \text{ bytes} = 2.052.000,000 \text{ bytes}$	$400.000 \times 5.121 \text{ bytes} = 2.048.400.000 \text{ bytes}$	$409 \times 5.000.001 \text{ bytes} = 2.045.000.409 \text{ bytes}$
AT = 2	$4.000.000 \times 516 \text{ bytes} = 2.064.000.000 \text{ bytes}$	$400.000 \times 5.124 \text{ bytes} = 2.049.600.000 \text{ bytes}$	$409 \times 5.000.004 \text{ bytes} = 2.045.001.636 \text{ bytes}$
AT = 3	$4.000.000 \times 519 \text{ bytes} = 2.076.000.000 \text{ bytes}$	$400.000 \times 5.127 \text{ bytes} = 2.050.800.000 \text{ bytes}$	$409 \times 5.000.007 \text{ bytes} = 2.045.002.863 \text{ bytes}$
AT = 4	$4.000.000 \times 522 \text{ bytes} = 2.088.000.000 \text{ bytes}$	$400.000 \times 5.130 \text{ bytes} = 2.052.000.000 \text{ bytes}$	$409 \times 5.000.010 \text{ bytes} = 2.045.004.090 \text{ bytes}$
AT = 5	$4.000.000 \times 525 \text{ bytes} = 2.100.000.000 \text{ bytes}$	$400.000 \times 5.133 \text{ bytes} = 2.053.200.000 \text{ bytes}$	$409 \times 5.000.013 \text{ bytes} = 2.045.005.317 \text{ bytes}$

Table 2: Μνήμη για σταθερό αριθμό αντικειμένων ανάλογα με το μέγεθος.

Ποσοστό Αύξησης Μνήμης για σταθερό αριθμό αντικειμένων ανάλογα με το μέγεθος			
Αντικείμενα που χωράνε σε 2,048,000,000 bytes	Μικρά αντικείμενα	Μεσαία αντικείμενα	Μεγάλα αντικείμενα
AT = 1	$\frac{(2,052,000,000 - 2,048,000,000)}{2,048,000,000} \times 100 = 0.195\%$	$\frac{(2,048,400,000 - 2,048,000,000)}{2,048,000,000} \times 100 = 0.0195\%$	$\frac{(2,045,000,409 - 2,045,000,000)}{2,045,000,000} \times 100 = 0.00002\%$
AT = 2	$\frac{(2,064,000,000 - 2,048,000,000)}{2,048,000,000} \times 100 = 0.781\%$	$\frac{(2,049,600,000 - 2,048,000,000)}{2,048,000,000} \times 100 = 0.0781\%$	$\frac{(2,045,001,636 - 2,045,000,000)}{2,045,000,000} \times 100 = 0.00008\%$
AT = 3	$\frac{(2,076,000,000 - 2,048,000,000)}{2,048,000,000} \times 100 = 1.367\%$	$\frac{(2,050,800,000 - 2,048,000,000)}{2,048,000,000} \times 100 = 0.1367\%$	$\frac{(2,045,002,863 - 2,045,000,000)}{2,045,000,000} \times 100 = 0.00014\%$
AT = 4	$\frac{(2,088,000,000 - 2,048,000,000)}{2,048,000,000} \times 100 = 1.953\%$	$\frac{(2,052,000,000 - 2,048,000,000)}{2,048,000,000} \times 100 = 0.1953\%$	$\frac{(2,045,004,090 - 2,045,000,000)}{2,045,000,000} \times 100 = 0.0002\%$
AT = 5	$\frac{(2,100,000,000 - 2,048,000,000)}{2,048,000,000} \times 100 = 2.539\%$	$\frac{(2,053,200,000 - 2,048,000,000)}{2,048,000,000} \times 100 = 0.2539\%$	$\frac{(2,045,005,317 - 2,045,000,000)}{2,045,000,000} \times 100 = 0.00026\%$

Table 3: HLRU - Ποσοστό Αύξησης Μνήμης συναρτήσει μεγέθους.

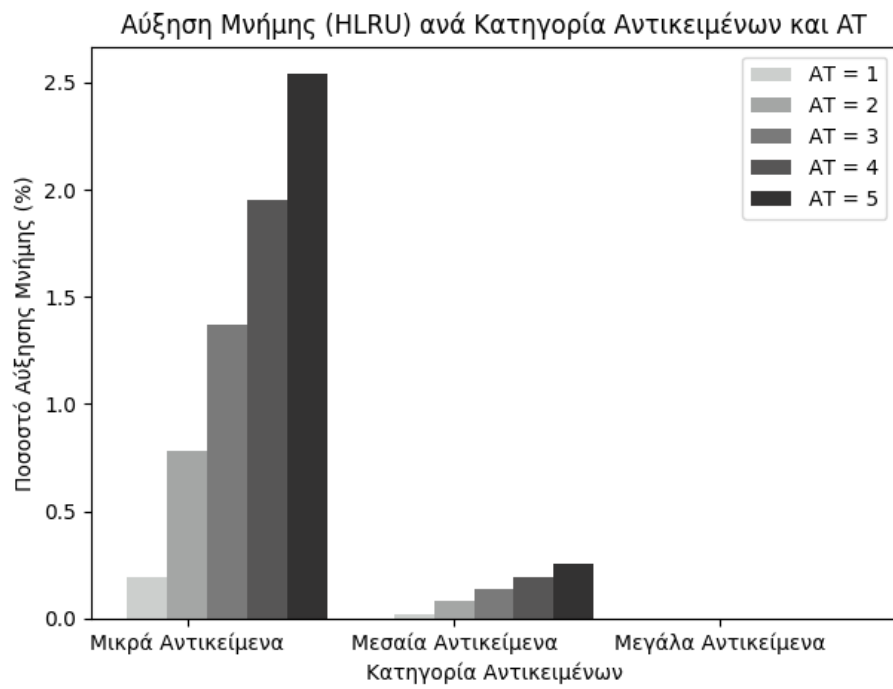


Figure 28: HLRU - Ποσοστό Αύξησης Μνήμης για σταθερό αριθμό αντικειμένων συναρτήσει μεγέθους.

Κεφάλαιο 5: Παρουσίαση και Ανάλυση Αποτελεσμάτων

Σε αυτήν την ενότητα, θα προβούμε σε σύγκριση, ανάλυση και σχολιασμό των σεναρίων που είδαμε στο προηγούμενο κεφάλαιο για την αξιολόγηση της προτεινόμενης μεθόδου αντικατάστασης δεδομένων **HLRU** (Redis Optis) συγκριτικά με τον παραδοσιακό αλγόριθμο **LRU** (Classic Redis).

Θα ασχοληθούμε αρχικά με κάποιες παρατηρήσεις που είναι κοινές ανάμεσα σε όλα τα πειράματα. Στη συνέχεια, θα ασχοληθούμε με την ανάλυση κάθε σεναρίου σύγκρισης μεταξύ HLRU και LRU ξεχωριστά. Θα αναλύσουμε τη σημασία των διαφορετικών τιμών του δυναμικού πίνακα AT, που χρησιμοποίησε ο αλγόριθμος του HLRU. Βάσει κριτηρίων, θα προτείνουμε και τιμές για το μέγεθος του πίνακα AT.

5.1 Παρατηρήσεις για όλα τα Σενάρια

Έχοντας τρέξει για κάθε σενάριο, επαρκή αριθμό από πειράματα, μπορούμε με ασφάλεια να βγάλουμε κάποια πρώτα πορίσματα.

5.1.1 Διαφοροποιήσεις Πειραμάτων για Ίδια Σενάρια

Από όλα τα σενάρια που τρέξαμε, παρατηρώντας και όλα τα πειράματα, παρατηρούμε πως ποτέ αυτά δεν είναι ίδια.

Αυτό οφείλεται στην παράμετρο *maxmemory-samples*, που όπως έχει ήδη περιγραφεί, η Redis εκτελεί τυχαία δειγματοληψία τόσων αντικειμένων όσων ορίζει αυτή η παράμετρος. Εφόσον, λοιπόν, κάθε φορά γίνεται τυχαία επιλογή, όπως είναι αυτονόητο, μπορεί να διαγραφούν περισσότερα ή λιγότερα δεδομένα από τα σημαντικά.

Αντιλαμβανόμαστε πως αν αυτή η παράμετρος ήταν ίση με τον αριθμό των αντικειμένων στη cache, τότε και μόνο τότε τα πειράματα θα έβγαιναν ίδια, καθώς τότε ο αλγόριθμος θα μπορούσε να αξιολογήσει ποιο είναι πραγματικά το λιγότερο χρήσιμο αντικείμενο της μνήμης.

5.1.2 Χρήση της CPU

Περί της, επίσης, χρήσης της CPU και στους δύο αλγορίθμους, παρατηρείται ότι η χρήση της αυξάνεται καθώς γίνονται αντικαταστάσεις. Αυτό είναι λογικό, καθώς σε εκείνη τη φάση εκτελούνται οι διαδικασίες αξιολόγησης των αντικειμένων που πρέπει να

απομακρυνθούν. Συγκριτικά με τους δύο αλγόριθμους, δεν παρατηρείται κάποια αξιοσημείωτη διαφορά.

5.2 Αποτελέσματα - Random Σενάριο

Στα πειράματα αυτά παρατηρούμε ότι δεν έχουμε διακυμάνσεις στις τιμές των Cache Misses. Η HLRU έχει {1, 2, 1, 1, 0, 0} cache misses, ενώ η LRU έχει {0, 1, 0, 2, 1, 1} cache misses, ενώ και οι 2 έχουν 5 συνολικά cache misses. Στα πρώτα τρία πειράματα έχουμε την *HLRU* να έχει περισσότερα Cache Misses, ενώ στα τρία τελευταία λιγότερα σε σχέση με την *LRU*. Συνεπώς, οι **δύο αλγόριθμοι αποδίδουν το ίδιο**.

5.3 Αποτελέσματα - Σενάριο Ευνοϊκό σε LRU με $f > 1$

Στο πολύ συγκεκριμένο αυτό σενάριο, που η διαμόρφωση της αξίας των αντικειμένων δεν συνάδει με το τελικό μοτίβο ανάκτησης, η HLRU έχει περισσότερα Cache Misses συγκριτικά με την LRU, και συγκεκριμένα:

Σενάριο Ευνοϊκό σε LRU με $f > 1$		
Εκτέλεση	HLRU Cache Misses	LRU Cache Misses
1η	8	5
2η	6	5
3η	7	5
Σύνολο	21	15
Ποσοστό Αύξησης CM	$\left(\frac{21 - 15}{15}\right) \times 100 \cong 40\%$	

Table 4: Σενάριο Ευνοϊκό σε LRU με $f > 1$ - Ποσοστό Αύξησης Μνήμης

Από τους παραπάνω υπολογισμούς συμπεραίνουμε ότι ο η HLRU έχει 21 cache misses συνολικά, ενώ η LRU έχει 15 στο σύνολο των 3 εκτελέσεων. Επομένως, η **HLRU έχει κατά μέσο όρο 40% περισσότερες αστοχίες** εν συγκρίσει με τον κλασικό **LRU** στο συγκεκριμένο σενάριο.

5.4 Αποτελέσματα - Σενάριο Ευνοϊκό σε LRU με $f = 1$

Στο σενάριο αυτό, στην ουσία η συχνότητα δεν έχει επίδραση στην αξιολόγηση των εντικειμένων για την HLRU, καθώς όλα τα αντικείμενα έχουν $f = 1$. Οπότε ο αλγόριθμος «εκφυλίζεται» σε LRU.

Από τα γραφήματα, παρατηρούμε πως η HLRU έχει {5, 3, 3, 3, 3} cache misses (17 συνολικά), ενώ η LRU έχει {2, 2, 2, 7, 7} cache misses (19 συνολικά).

Η διαφορά των cache misses είναι αμελητέα, και θεωρούμε οι **δύο αλγόριθμοι αποδίδουν το ίδιο**.

5.5 Αποτελέσματα - Σενάριο Hot & Cold

Στο σενάριο αυτό που παίζει ρόλο κα η χρονική εγγύτητα, αλλά και η συχνότητα, η απόδοση της HLRU είναι αισθητά καλύτερη από την LRU.

Σενάριο Hot & Cold		
Εκτέλεση	HLRU Cache Misses	LRU Cache Misses
1η	15	27
2η	0	7
3η	8	14
4η	8	17
Σύνολο	31	65
Ποσοστό Αύξησης CM	$\left(\frac{65 - 31}{31}\right) \times 100 \cong \mathbf{110\%}$	

Table 5: Σενάριο Hot & Cold - Ποσοστό Αύξησης Cache Misses

Η HLRU έχει συνολικά 31 cache misses στο σύνολο των 4 εκτελέσεων, ενώ η LRU έχει 65. Από τους παραπάνω υπολογισμούς συμπεραίνουμε ότι ο αλγόριθμος **LRU** κάνει **κατά μέσο όρο 110% περισσότερες αστοχίες** εν συγκρίσει με τον κλασικό **HLRU** στο συγκεκριμένο σενάριο. Αντίστοιχα, συμπεραίνουμε ότι ο **HLRU** έχει **52.3% λιγότερες αστοχίες** από τον LRU.

5.6 Αποτελέσματα - Σενάριο Σειριακής Σάρωσης

Στο σενάριο αυτό που παίζει ρόλο κα η χρονική εγγύτητα, αλλά και η συχνότητα, η απόδοση της HLRU είναι αισθητά καλύτερη από την LRU. Η LRU, εν αντιθέσει της HLRU, παραπλανήθηκε και επηρεάστηκε από τη σάρωση των αντικειμένων κατά τη διάρκεια του πειράματος, με αποτέλεσμα να μην αξιολογήσει σωστά τα αντικείμενα που είχε στη μνήμη της,

Όπως θα δούμε και από τους υπολογισμούς, η απόδοση της HLRU είναι αισθητά καλύτερη από την LRU.

Σενάριο Σειριακής Αναζήτησης		
Εκτέλεση	HLRU Cache Misses	LRU Cache Misses
1η	13	22
2η	11	20
3η	2	6
4η	2	3
Σύνολο	28	51
Ποσοστό Αύξησης CM	$\left(\frac{51 - 28}{28}\right) \times 100 \cong \mathbf{82.1\%}$	

Table 6: Σενάριο Σειριακής Σάρωσης - Ποσοστό Αύξησης Cache Misses

Από τους παραπάνω υπολογισμούς συμπεραίνουμε ότι ο αλγόριθμος **LRU** κάνει **κατά μέσο όρο 82.1% περισσότερες αστοχίες** εν συγκρίσει με τον κλασικό **HLRU** στο συγκεκριμένο σενάριο. Αντίστοιχα, συμπεραίνουμε ότι ο **HLRU** έχει **45.1% λιγότερες αστοχίες** από τον LRU.

5.7 Αποτελέσματα - HLRU (AT = {1, 2, 3, 4, 5})

Παρατηρώντας τα γραφήματα, συμπεραίνουμε πως σε ένα ανταγωνιστικό σενάριο, το λεπτομερές ιστορικό έχει μεγάλη αξία για τη σωστή αξιολόγηση της σημασίας των αντικειμένων. Όσο πιο λεπτομερές είναι το ιστορικό, δηλαδή όσες περισσότερες τιμές κρατάμε στον πίνακα AT, τόσο φαίνεται να αποδίδει καλύτερα ο αλγόριθμος, καθώς ο αριθμός των αστοχιών μειώνεται.

Στον πίνακα που ακολουθεί, συγκεντρώνονται ο συνολικός αριθμός αστοχιών και ο μέσος όρος αστοχιών, καθώς και το χειρότερο και το καλύτερο σενάριο αύξησης μνήμης για κάθε τιμή AT, όπως υπολογίστηκαν στο προηγούμενο Κεφάλαιο (Table 3).

HLRU					
Εκτέλεση	HLRU AT = 1	HLRU AT = 2	HLRU AT = 3	HLRU AT = 4	HLRU AT = 5
1η	3	0	3	0	0
2η	5	1	1	0	1
3η	2	4	0	0	1
4η	2	0	1	0	0
5η	5	2	0	1	0
6η	6	1	1	2	0
7η	6	1	1	0	0
8η	3	5	0	1	0
9η	4	2	0	2	1
10η	3	2	1	0	0
Σύνολο	39	18	8	6	3
Μέσος Όρος Cache Miss	$\left(\frac{39}{10}\right) = 3.9$	$\left(\frac{18}{10}\right) = 1.8$	$\left(\frac{8}{10}\right) = 0.7$	$\left(\frac{6}{10}\right) = 0.6$	$\left(\frac{3}{10}\right) = 0.3$
Ποσοστό Αύξησης Μνήμης - Worst Case Scenario	0.195%	0.781%	1.367%	1.953%	2.539%
Ποσοστό Αύξησης Μνήμης - Best Case Scenario	0.00002%	0.00008%	0.00014%	0.0002%	0.00026%

Table 7: HLRU (AT = {1, 2, 3, 4, 5}) - Συγκεντρωτικός Πίνακας

Από τον παραπάνω πίνακα, μπορούμε να δούμε πως τη μεγαλύτερη ακρίβεια την πετυχαίνουμε όσο μεγαλώνει το AT, όσο δηλαδή μεγαλύτερη ιστορικό έχουμε για τα δεδομένα. Όμως, αυτό έρχεται με το τίμημα (trade-off) της χρήσης περισσότερης μνήμης.

Η επιλογή της βέλτιστης τιμής AT εξαρτάται από παράγοντες όπως:

- Οι διαθέσιμοι πόροι μνήμης του συστήματος,
- Τα πρότυπα πρόσβασης στα δεδομένα,
- Οι επιδόσεις που απαιτούνται από την εφαρμογή.

Ένας κρίσιμος περιοριστικός παράγοντας είναι η τυχειότητα που εισάγει το *maxmemory-samples* της Redis, το οποίο χρησιμοποιεί δειγματοληψία για την επιλογή των αντικειμένων προς διαγραφή. Αυτό προσθέτει ένα επίπεδο *τυχειότητας* και *αβεβαιότητας*, καθώς δεν εξασφαλίζει ότι πάντα θα διατηρούνται τα πλέον χρήσιμα δεδομένα στη μνήμη.

Ως εκ τούτου, η αύξηση του AT πέρα από το 5 δεν αναμένεται να προσφέρει σημαντική βελτίωση, αφού η πιθανότητα να επιλεγούν τα πλέον χρήσιμα δεδομένα δεν αυξάνεται γραμμικά με το AT. Σε πρακτικές συνθήκες, η βέλτιστη επιλογή θα πρέπει να γίνεται βάσει δοκιμών και ανάλυσης της πραγματικής συμπεριφοράς της εφαρμογής.

5.7.1 Αξιολόγηση και Use Cases HLRU ($AT = \{1, 2, 3, 4, 5\}$)

Στον παρακάτω πίνακα, παρουσιάζονται οι συνθήκες στις οποίες κάθε τιμή AT είναι κατάλληλη, μαζί με τα πλεονεκτήματα και τα μειονεκτήματά της.

Αξιολόγηση HLRU βάσει AT			
Τιμή AT	Πότε είναι χρήσιμη;	Πλεονεκτήματα	Μειονεκτήματα
Minimum $AT = \{1, 2\}$	■ Περιορισμένη μνήμη ■ Όταν τα δεδομένα δεν έχουν υψηλή πιθανότητα να ξαναζητηθούν (low locality) ■ Όταν η cache πρέπει να έχει πάντα τα πιο πρόσφατα δεδομένα (real-time updates)	■ Χαμηλότερη χρήση μνήμης (συγκριτικά)	■ Υψηλότερο ποσοστό cache misses ■ Μικρότερο caching όφελος σε workloads με υψηλό locality
Medium	■ Γενικά ισορροπημένη επιλογή ■ Όταν θέλουμε σημαντική μείωση cache misses χωρίς υπερβολική χρήση μνήμης	■ Μειώνει δραστικά τα cache misses	■ Ελαφρώς μεγαλύτερη κατανάλωση μνήμης σε σχέση με $AT=1$
Maximum $AT = 5$	■ Η μνήμη δεν είναι περιοριστικός παράγοντας ■ Όταν το workload έχει υψηλό locality (τα ίδια δεδομένα ζητούνται ξανά & ξανά) ■ Όταν ανεχόμαστε ελάχιστα cache misses και χρειαζόμαστε χαμηλό latency	■ Λιγότερα cache misses ■ Ιδανικό για high-performance workloads	■ Υψηλότερη χρήση μνήμης ■ Περισσότερη χρήση μνήμης σε μη ανταγωνιστικά περιβάλλοντα ή workloads με χαμηλό locality

Table 8: Αξιολόγηση και Use Cases HLRU βάσει AT

5.8 Συμπεράσματα

Λαμβανομένου υπ'όψιν όλων των παρατηρήσεων και των αποτελεσμάτων που παρουσιάστηκαν στις προηγούμενες ενότητες, καταλήγουμε στο συμπέρασμα ότι ο προτεινόμενος αλγόριθμος HLRU επιδεικνύει καλές επιδόσεις, ειδικά σε περιβάλλοντα όπου τόσο η χρονική εγγύτητα (recency) όσο και η συχνότητα πρόσβασης (frequency) παίζουν ρόλο στη διατήρηση των δεδομένων στη μνήμη. Συγκεκριμένα, από τα πειράματα που εκτελέστηκαν, προέκυψαν τα εξής ευρήματα:

- Σενάρια τυχαίων προσβάσεων & απλοϊκών μοτίβων αναζήτησης: Σε περιβάλλοντα όπου οι προσβάσεις στα δεδομένα είναι εντελώς τυχαίες ή ακολουθούν πολύ απλά μοτίβα, ο HLRU παρουσίασε επιδόσεις αντίστοιχες με τον LRU, καθώς η ιστορική πληροφορία δεν προσέφερε ουσιαστικό πλεονέκτημα στη διαχείριση της μνήμης.
- Σενάρια όπου κυριαρχεί η χρονική εγγύτητα (recency-only): Όταν η χρονική εγγύτητα είναι ο μόνος παράγοντας, δηλαδή, όταν τα πιο πρόσφατα δεδομένα είναι πάντα τα πιο σημαντικά για την εφαρμογή, ο LRU υπερέχει του HLRU.
- Σενάρια όπου η συχνότητα και η χρονική εγγύτητα συνδυάζονται: Ο HLRU αποδείχθηκε σαφώς ανώτερος σε περιβάλλοντα όπου τα δεδομένα επανεμφανίζονται, αλλά ταυτόχρονα η χρονική εγγύτητα εξακολουθεί να έχει σημασία. Στα σενάρια αυτά εντάσσεται το Hot & Cold σενάριο και το Scan σενάριο.
- Σενάρια με μοτίβα πρόσβασης μεγάλου ανταγωνισμού: Η HLRU μπορεί να μειώσει δραστικά τα cache misses εν συγκρίσει με την LRU. Η τιμή AT έχει ορατά οφέλη στην απόδοση της cache.
 - Ο HLRU με τιμές $AT = \{1, 2, 3, 4, 5\}$ μειώνουν δραστικά τα cache misses, με τα cache misses να μειώνονται όσο αυξάνεται η τιμή του AT, άρα και της μνήμης.
 - Η αύξηση του AT πέρα από το $AT = 5$ δε θα παρείχε σημαντικά οφέλη, επιβεβαιώνοντας ότι η ιστορική πληροφορία έχει φθίνουσα απόδοση λόγω της τυχαιότητας που εισάγει το *maxmemory-samples*.

Κεφάλαιο 6: Περιορισμοί και Μελλοντική Έρευνα

Παρά τη βελτιωμένη απόδοση του αλγορίθμου HLRU σε σενάρια με υψηλό locality, υπάρχουν ορισμένοι περιορισμοί που πρέπει να ληφθούν υπ' όψιν:

6.1.1 Εκτέλεση σε Παραγωγικά Περιβάλλοντα

Στην παρούσα μελέτη, οι πειραματικές δοκιμές πραγματοποιήθηκαν τοπικά σε Εικονική Μηχανή γεγονός που περιορίζει την εξαγωγή συμπερασμάτων για την απόδοσή της HLRU σε μεγάλης κλίμακας συστήματα με πραγματικά δεδομένα. Αν και η HLRU δεν εμφανίζει κάποιο computational overhead, όπως είδαμε στα διαγράμματα, δεν έχει εξεταστεί η συμπεριφορά του HLRU σε διαφορετικά setups, όπως σε κατανεμημένα περιβάλλοντα με απομακρυσμένη μνήμη ή multi-node αρχιτεκτονικές.

Επομένως, για την πλήρη αξιολόγηση των μετρικών που είδαμε, απαιτείται περαιτέρω πειραματισμός σε μεγαλύτερης κλίμακας συστήματα, με διαφορετικές συνθήκες φόρτου και πιο απαιτητικές αρχιτεκτονικές.

6.1.2 Δυσκολία στην Επιλογή της Βέλτιστης Τιμής AT

Όπως είδαμε η ιδανική τιμή AT εξαρτάται από τα χαρακτηριστικά του συστήματος (περιορισμοί, μοτίβα πρόσβασης) και δεν υπάρχει μία μοναδική και καθολικά βέλτιστη τιμή. Ενώ η κατασκευή των ορίων των παραθύρων του ιστογράμματος είναι δυναμικά υπολογιζόμενα με χρήση μαθηματικών εννοιών όπως ο μέσος όρος και η τυπική απόκλιση, ο αλγόριθμος δεν αναγνωρίζει το μοτίβο εμφάνισης των δεδομένων και δεν είναι σε θέση να προσαρμόσει την τιμή AT βάσει αυτού. Αντίθετα, ενώ η τιμή μπορεί να αλλάξει on-the-fly, απαιτεί πειραματισμό μέχρι να βρεθεί η «σωστή» τιμή για το εκάστοτε σύστημα.

Επίσης, τα βάρη των παραθύρων που χρησιμοποιήθηκαν, επιδέχονται περισσότερου πειραματισμού και fine-tuning. Ενώ και αυτά είναι δυναμικά, και μπορούν επίσης να αλλάξουν κατά τη διάρκεια της λειτουργίας της Redis, δεν αλλάζουν με αυτοματοποιημένο τρόπο.

Σε μελλοντική μελέτη, η δημιουργία ενός Redis client, ο οποίος θα αναγνώριζε το μοτίβο εμφάνισης των αντικειμένων, θα ήταν μία πολύ σημαντική προσθήκη στην υλοποίησή μας. Η χρήση ενός τέτοιου client, θα συνεισέφερε στη σωστή επιλογή της τιμής του AT και των βαρών με αυτόματο τρόπο, καθώς και στην ρύθμιση αυτών αν το μοτίβο πρόσβασης διαφοροποιούνταν.

Κεφάλαιο 7: Επίλογος

Στα σύγχρονα υπολογιστικά συστήματα με όλο αυξανόμενες απαιτήσεις για αποδοτικότητα και ταχύτητα, η αποδοτική ανάκτηση δεδομένων αποτελεί κρίσιμο παράγοντα. Η κρυφή μνήμη, ούσα περιορισμένης χωρητικότητας, καθιστά απαραίτητη τη σωστή επιλογή και διαχείριση των δεδομένων που θα παραμείνουν σε αυτήν, και άρα τη σωστή επιλογή πολιτικής αντικατάστασης.

Στην παρούσα εργασία, προβήκαμε στην ανάπτυξη της Histogram-based LRU με σκοπό την ορθή αξιολόγηση της χρησιμότητας των δεδομένων, βασισμένη στη χρονική εγγύτητα και συχνότητα, μετριάζοντας τα προβλήματα που συνδέονται με την κλασική LRU. Μέσω της διενέργειας ενδελεχούς πειραματικής μελέτης σε διαφορετικά μοτίβα πρόσβασης, καταλήξαμε πως: (1) ο αλγόριθμος μας αποδίδει εφάμιλλα με την LRU σε LRU και Random σενάρια, και (2) επιτυγχάνει **52.3%** και **45.1%** μείωση των cache misses σε σενάρια Hot&Cold και Scanning αντίστοιχα.

Παρ'όλα τα θετικά αποτελέσματα, υπάρχει χώρος για βελτίωση, με μελλοντική έρευνα να συμπεριλαμβάνει τη δυναμική εύρεση και ορισμό της βέλτιστης τιμής AT για τη βελτιστοποίηση του trade-off μεταξύ ακρίβειας και χρήσης μνήμης.

Η συνολική μας έρευνα καταδεικνύει ότι η Histogram-based LRU (HLRU) αποτελεί μια βελτιωμένη εκδοχή της κλασικής LRU, ικανή να προσαρμόζεται σε ένα ευρύ φάσμα από locality-oriented και frequency-oriented use cases. Ο αλγόριθμος αυτός επιτρέπει τη λήψη πιο εύστοχων αποφάσεων σχετικά με τη σημασία των δεδομένων συμβάλλοντας στην ενίσχυση της απόδοσης της cache και εν τέλει όλου του συστήματος.

Βιβλιογραφία

- [1] Redis 2024. *Redis Eviction Policies*. Retrieved May 5, 2024 from <https://redis.io/docs/eviction/>
- [2] Redis 2024. *Redis LRU Cache*. Retrieved June 15, 2024 from <https://redis.io/glossary/lru-cache/>
- [3] Redis 2024. *Redis 7.2.4*. Retrieved March 16, 2024 from <http://download.redis.io/releases>
- [4] Thanasis Prioivos, Stathis Maroulis, Vana Kalogeraki (2019, October) “*ESCAPE: Elastic Caching for Big Data Systems*”.
- [5] Thanasis Prioivos, Stathis Maroulis, Vana Kalogeraki (2019, October) “*A Framework for Managing an Elastic Redis Cache*”.
- [6] Yuchen Wang, Junyao Yang, Zhenlin Wang (2023, October). “*Multi-Tenant In-memory Key-Value Cache Partitioning Using Efficient Random Sampling-Based LRU Model*”.
- [7] Yuchen Wang, Junyao Yang, Zhenlin Wang (2021, October). “*Efficient Modeling of Random Sampling-Based LRU Cache*”.
- [8] Yuchen Wang, Junyao Yang, Zhenlin Wang (2020, September). “*Dynamically Configuring LRU Replacement Policy in Redis*”.
- [9] Cheng Pan, Xiaolin Wang, Yingwei Luo, Zhenlin Wang (2021, May). “*Penalty- and Locality-aware Memory Allocation in Redis Using Enhanced AET*”.
- [10] Cheng Pan, Xiaolin Wang, Yingwei Luo, Zhenlin Wang (2019, November). “*pRedis: Penalty and Locality Aware Memory Allocation in Redis*”.
- [11] Cheng Pan, Xiaolin Wang, Yingwei Luo, Zhenlin Wang (2018, October). “*PACE: Penalty aware cache modeling with enhanced AET*”.
- [12] Mei Li, Hong-Jun Zhang, Yan-Jun Wu, Chen Zhao (2017, January). “*MemSC: A Scan-Resistant and Compact Cache Replacement Framework for Memory-Based Key-Value Cache Systems*”.
- [13] Conglong Li, Alan L. Cox (2015, April). “*GD-wheel: A cost-aware replacement policy for key-value stores*”.

Appendix - Code

```
typedef struct {
    unsigned access_time: LRU_BITS; // Use exactly LRU_BITS for each entry
} AccessTime;

struct redisObject {
    unsigned type:4;
    unsigned encoding:4;
    // unsigned lru:LRU_BITS; /* LRU time (relative to global lru_clock) */
    int refcount;
    void *ptr;
    AccessTime access_times[HISTORY_SIZE]; /* Dynamic Array */
    unsigned num_accesses:ACCESS_BITS; /* Number of accesses */
};
```

Figure 29: Code - Δημιουργία HLRU variables στο redisObject (access_times and num_accesses)

```
# Eviction policy: allkeys-lru-histogram
maxmemory-policy allkeys-lru-histogram
# Define how many access times should be kept for each item.
## This should be a value between 1 and 5.
history-count 5
# Defines the weights of the windows created during the eviction phase for building the histogram.
## A descending array of numbers; the higher values indicate that the first windows
## (and hence the most recently used items) are more important.
histogram-window-weights 0.4 0.3 0.2 0.15
```

Figure 30: Code - Δυναμικό Configuration (history-count, histogram-window-weights)

```

void calculateStatistics(dictEntry **samples, int sample_count, unsigned long long *global_min,
                        unsigned long long *global_max, double *mean, double *stddev) {
    unsigned long long current_time = LRU_CLOCK(), sum = 0, sum_of_squares = 0, sum_of_accesses = 0, count = 0;
    *global_min = ULLONG_MAX;
    *global_max = 0;

    for (int j = 0; j < sample_count; j++) {
        dictEntry *de = samples[j];
        sds key = dictGetKey(de);
        robj *o = dictGetVal(de);
        unsigned long long obj_min = ULLONG_MAX;
        unsigned long long obj_max = 0;

        for (unsigned int i = 0; i < o->num_accesses; i++) {
            unsigned long long age = current_time - o->access_times[i];
            // Update object-level min and max & Update global min and max
            if (age < obj_min) obj_min = age;
            if (age > obj_max) obj_max = age;
            if (age < *global_min) *global_min = age;
            if (age > *global_max) *global_max = age;

            sum += age;
            sum_of_squares += age * age;
            count++;
        }
    }
    // Compute mean and standard deviation
    if (count > 0) {
        *mean = (double)sum / count;
        *stddev = sqrt((double)sum_of_squares / count - (*mean) * (*mean));
    } else {
        *mean = 0.0;
        *stddev = 0.0;
    }
}

```

Figure 31: Code – Υπολογισμός Στατιστικών Δεδομένων (Mean, Standard Deviation)

```

void defineWindowBoundaries(double *window_boundaries, unsigned long long global_min,
                            unsigned long long global_max, double mean, double stddev) {

    window_boundaries[0] = global_min;

    window_boundaries[1] = fmax(global_min, mean - stddev); // No less than global_min
    window_boundaries[2] = fmax(global_min, fmin(global_max, mean)); // Clamped between min and max
    window_boundaries[3] = fmin(global_max, mean + stddev); // No more than global_max

    window_boundaries[4] = global_max;
}

```

Figure 32: Ορισμός Δυναμικών Παραθύρων Ιστογράμματος.

```

double estimateObjectEvictionScore(robj *o, double *window_boundaries, sds key) {
    unsigned long long current_time = LRU_CLOCK();
    const double WINDOW_WEIGHTS[NUM_WINDOWS] = {
        WINDOW_WEIGHT_1,
        WINDOW_WEIGHT_2,
        WINDOW_WEIGHT_3,
        WINDOW_WEIGHT_4,
        WINDOW_WEIGHT_5
    };

    int window_counts[NUM_WINDOWS+1] = {0};
    double score = 0.0;

    // Loop through each access time stored for the object
    for (unsigned int i = 0; i < o->num_accesses; i++) {
        unsigned long long age = current_time - o->access_times[i];
        // Determine the window each access time belongs to based on dynamic boundaries
        for (int j = 1; j < NUM_WINDOWS+1; j++) {
            if (age <= window_boundaries[j]) {
                window_counts[j]++;
                break; // Exit the loop after determining the appropriate window
            }
        }
    }

    // Calculate weighted score
    for (int i = 1; i < NUM_WINDOWS; i++) {
        score += (double>window_counts[i] * WINDOW_WEIGHTS[i];
    }

    double tie_breaker = 0.01 * o->num_accesses; // Small tie-breaking factor based on num_accesses
    score += tie_breaker;
    return -score;
}

```

Figure 33: Code - Υπολογισμός τελικού score.